Data-Oriented Debugging Across Place and Time

Jeffrey Tao, Xiaorui Liu, Ryan Marcus, Andrew Head

UIST 2026

Do you use print debugging?

Do you use print debugging?

Do you use breakpoint debugging?

Do you use print debugging?

Do you use breakpoint debugging?

Why?

Data-Oriented Debugging

This is an unconventional talk!

This is an unconventional talk!

1. Conceptual Lens: Data-Oriented Debugging
2. Observing Data-Oriented Debugging in studies
3. Future Data-Oriented Directions

This is an unconventional talk!

1. Conceptual Lens: Data-Oriented Debugging
2. Observing Data-Oriented Debugging in studies
3. Future Data-Oriented Directions

There will be no AI! But ask during Q&A 😊

Calculator Example

```
def evaluate(node):
    if isinstance(node, Num):
        return node.value
    left = evaluate(node.left)
    right = evaluate(node.right)
    match node.op:
        case "+": result = left + right
        case "*": result = left * right
        case "^": result = left * right
     return result
```

Calculator Example

```
def evaluate(node):
    if isinstance(node, Num):
        return node.value
    left = evaluate(node.left)
    right = evaluate(node.right)
    match node.op:
        case "+": result = left + right
        case "*": result = left * right
        case "^": result = left * right
     return result
```

Terminal

```
>>> calculate("2 + 3 * 4 ^ 2")
26
```

Calculator Example

```
1 def evaluate(node):
2     if isinstance(node, Num):
3         return node.value
4     left = evaluate(node.left)
5     right = evaluate(node.right)
6     match node.op:
7         case "+": result = left + right
8         case "*": result = left * right
9         case "^": result = left * right
10     return result
```

Terminal

```
>>> calculate("2 + 3 * 4 ^ 2")
26
```

"Order of operations is incorrect?"

Calculator Example

```
1 def evaluate(node):
2     if isinstance(node, Num):
3         return node.value
4     left = evaluate(node.left)
5     right = evaluate(node.right)
6     match node.op:
7         case "+": result = left + right
8         case "*": result = left * right
9         case "^": result = left * right
10     print(node.op)
11     return result
```

Terminal

```
>>> calculate("2 + 3 * 4 ^ 2")
26
```

"Order of operations is incorrect?"

Calculator Example

```
1 def evaluate(node):
2     if isinstance(node, Num):
3         return node.value
4     left = evaluate(node.left)
5     right = evaluate(node.right)
6     match node.op:
7         case "+": result = left + right
8         case "*": result = left * right
9         case "^": result = left * right
10     print(node.op)
11     return result
```

Terminal

```
>>> calculate("2 + 3 * 4 ^ 2")
^
*
+
26
```

"Order of operations is incorrect?"

Calculator Example

```
1 def evaluate(node):
2     if isinstance(node, Num):
3         return node.value
4     left = evaluate(node.left)
5     right = evaluate(node.right)
6     match node.op:
7         case "+": result = left + right
8         case "*": result = left * right
9         case "^": result = left * right
10     print(node.op)
11     return result
```

Terminal

```
>>> calculate("2 + 3 * 4 ^ 2")
^
*
+
26
```

"Nope!"

"Order of operations is incorrect?"

Calculator Example

```
1 def evaluate(node):
2     if isinstance(node, Num):
3         return node.value
4     left = evaluate(node.left)
5     right = evaluate(node.right)
6     match node.op:
7         case "+": result = left + right
8         case "*": result = left * right
9         case "^": result = left * right
10     print(node.op)
11     return result
```

Terminal

```
>>> calculate("2 + 3 * 4 ^ 2")
^
*
+
26
```

"Nope!"

"Order of operations is incorrect?"

① Data Need

Calculator Example

```
1 def evaluate(node):
2     if isinstance(node, Num):
3         return node.value
4     left = evaluate(node.left)
5     right = evaluate(node.right)
6     match node.op:
7         case "+": result = left + right
8         case "*": result = left * right
9         case "^": result = left * right
10     print(node.op)          ② Query
11     return result
```

Terminal

```
>>> calculate("2 + 3 * 4 ^ 2")
^
*
+
26
```

"Nope!"

"Order of operations is incorrect?"

① Data Need

Calculator Example

```
1 def evaluate(node):
2     if isinstance(node, Num):
3         return node.value
4     left = evaluate(node.left)
5     right = evaluate(node.right)
6     match node.op:
7         case "+": result = left + right
8         case "*": result = left * right
9         case "^": result = left * right
10     print(node.op)
11     return result
```

② Query

Terminal

```
>>> calculate("2 + 3 * 4 ^ 2")
^
*
+
26
```

③ **Interpret output**

"Nope!"

"Order of operations is incorrect?"

① Data Need

Calculator Example

```
1 def evaluate(node):
2     if isinstance(node, Num):
3         return node.value
4     left = evaluate(node.left)
5     right = evaluate(node.right)
6     match node.op:
7         case "+": result = left + right
8         case "*": result = left * right
9         case "^": result = left * right
10     print(node.op)
11     return result
```

② Query

Terminal

```
>>> calculate("2 + 3 * 4 ^ 2")
^
*
+
26
```

③ Interpret output

"Nope!"

④ Update mental model

"Order of operations is incorrect?"

① Data Need

Calculator Example

```
1 def evaluate(node):
6     match node.op:
7         case "+": result = left + right
8         case "*": result = left * right
9         case "^": result = left * right
10    print(node.op)
11    return result
```

② Query

Takeaway:
Think about data needs and how queries are specified

Terminal

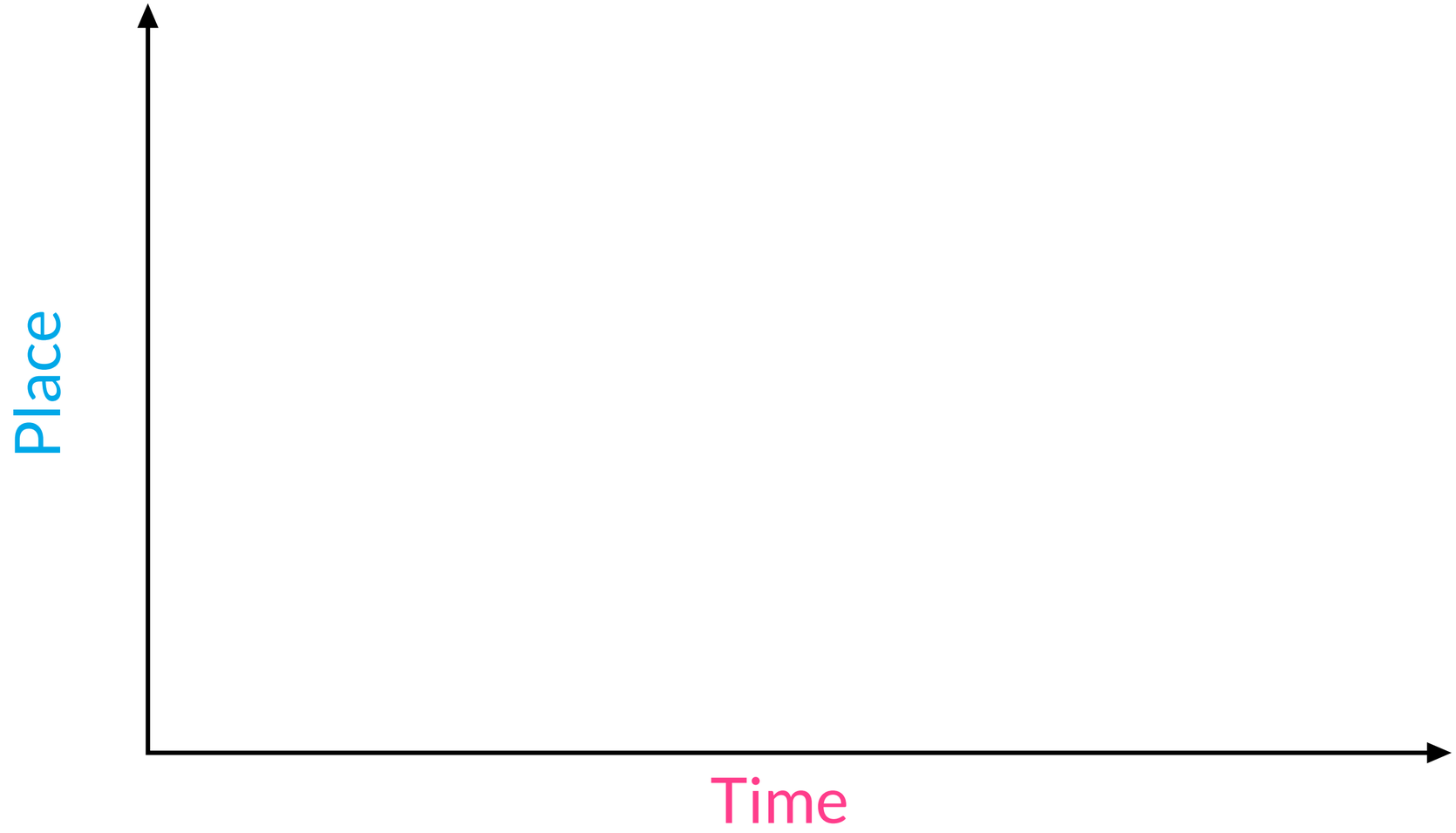
"Order of operations is incorrect?"

① Data Need

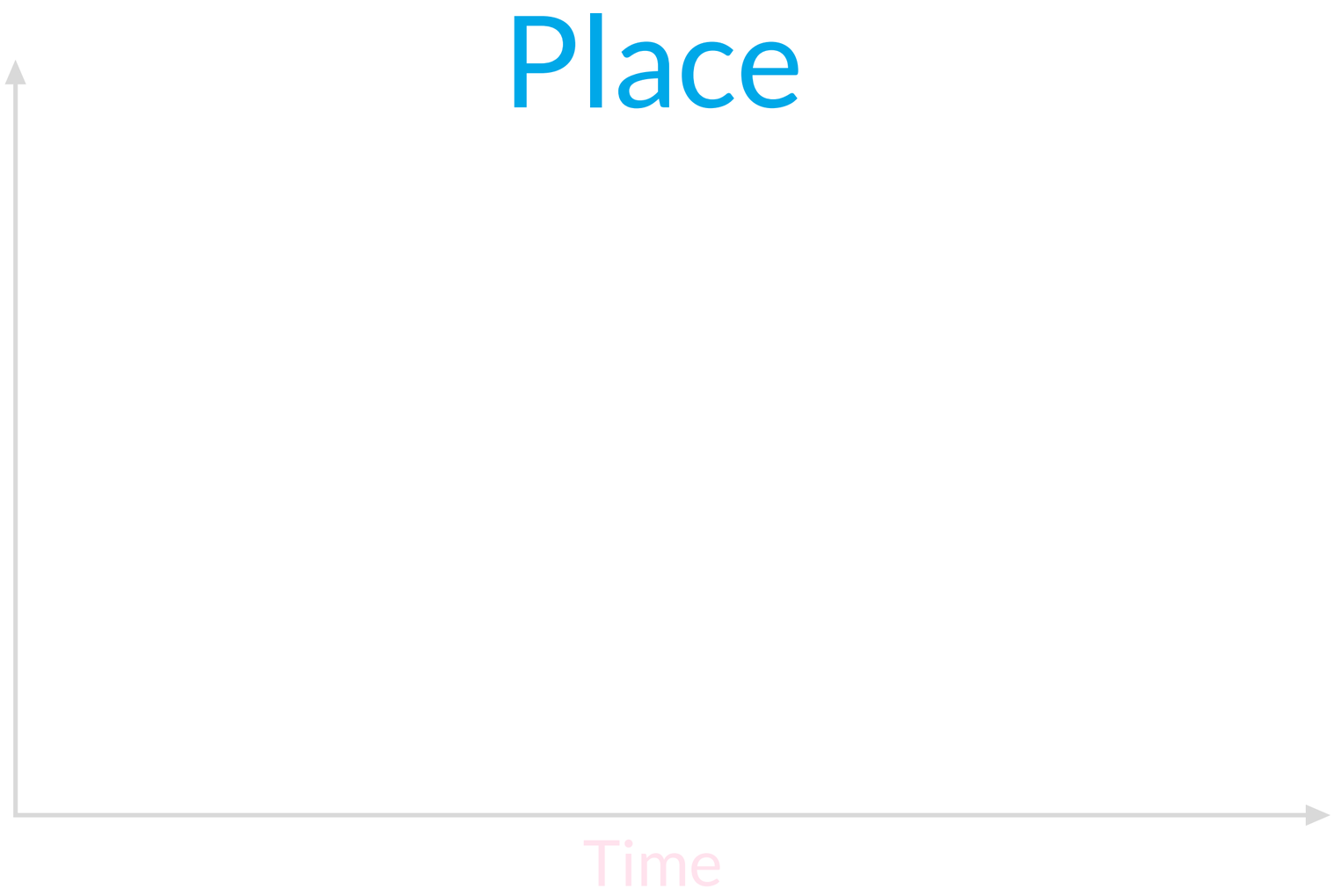
Place & Time

```
def evaluate(node):
    if isinstance(node, Num):
        return node.value
    left = evaluate(node.left)
    right = evaluate(node.right)
    match node.op:
        case "+": result = left + right
        case "*": result = left * right
        case "^": result = left * right
     return result
```

Place & Time



Place & Time



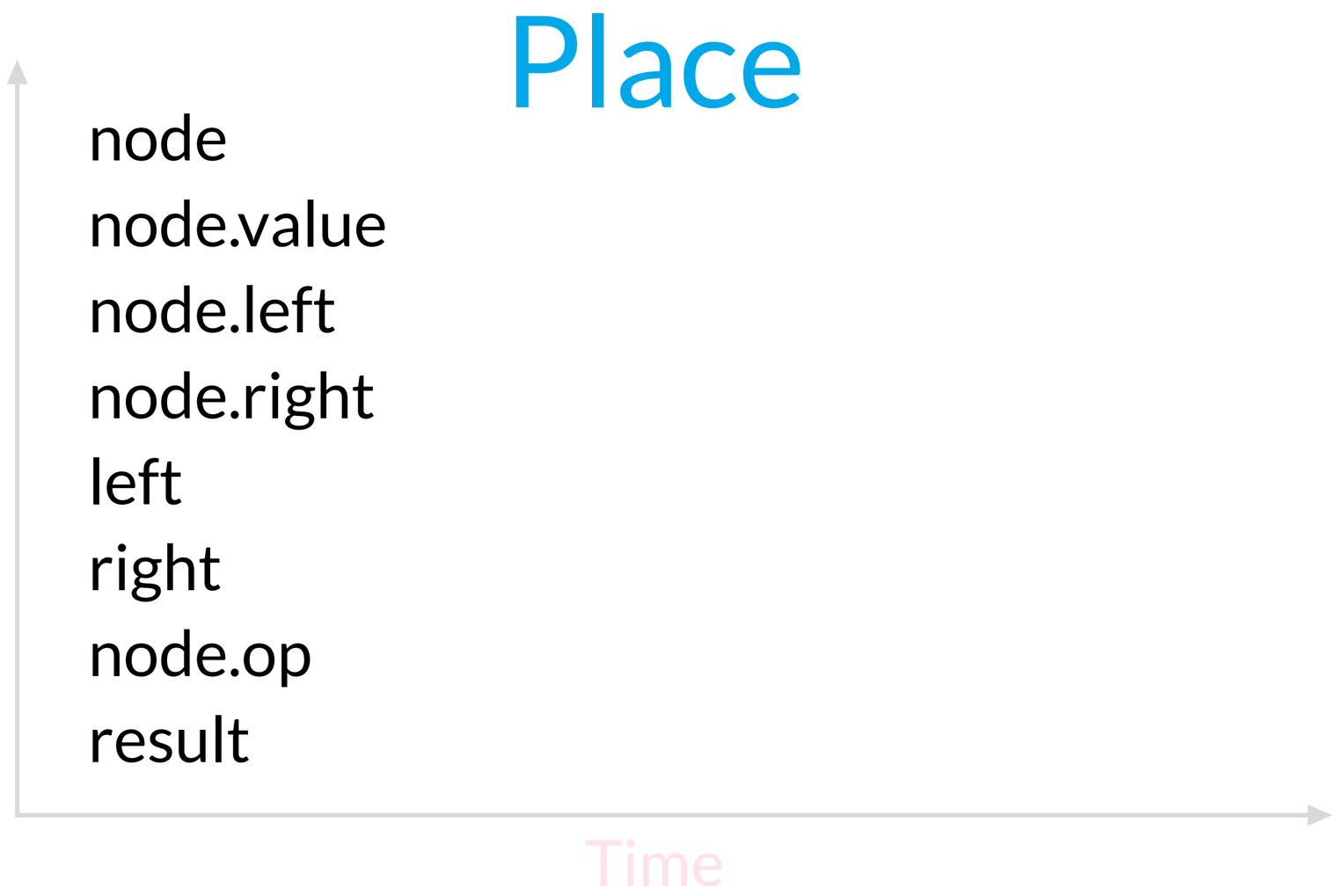
Place & Time

Place

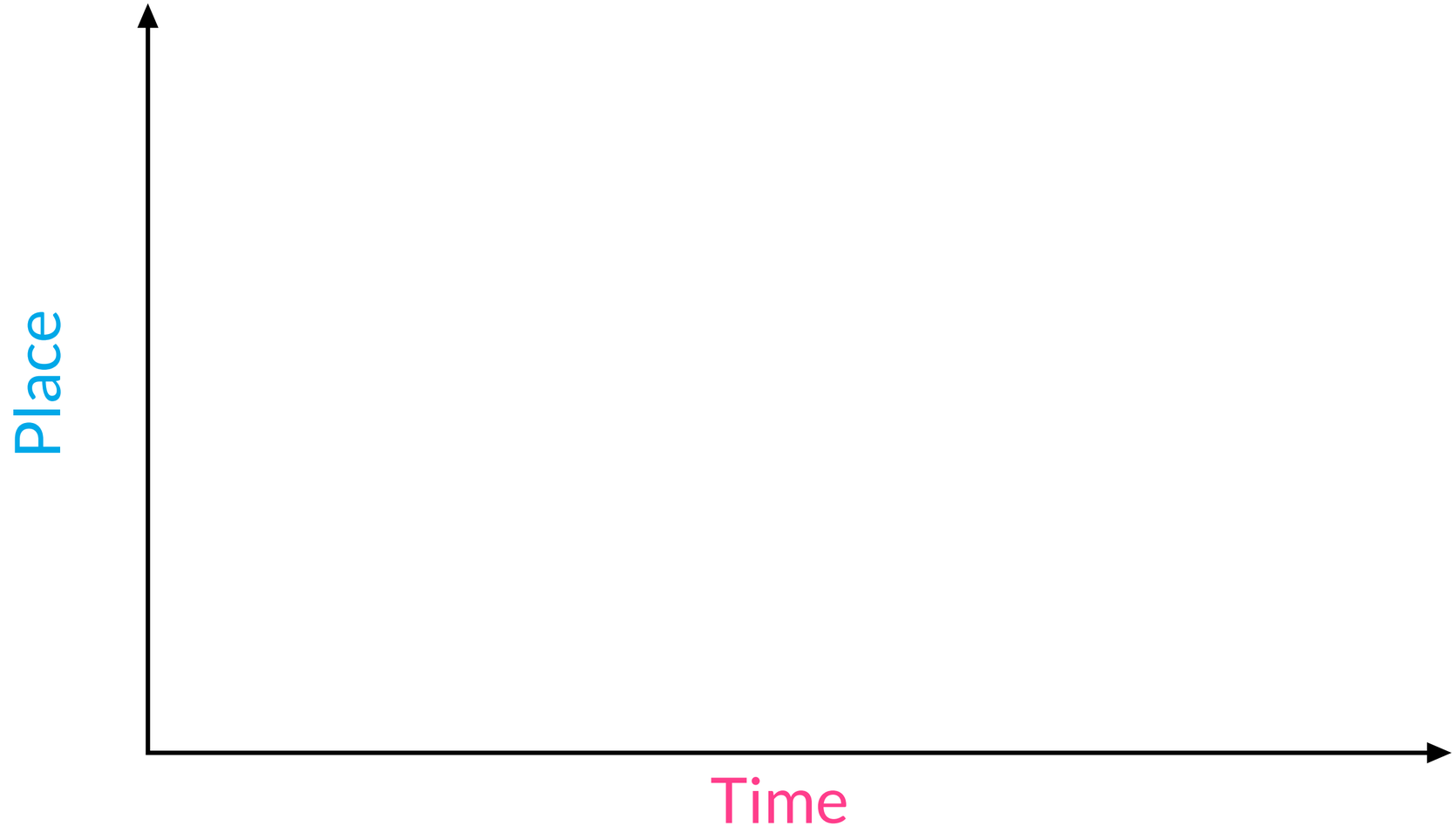
```
def evaluate(node):
    if isinstance(node, Num):
        return node.value
    left = evaluate(node.left)
    right = evaluate(node.right)
    match node.op:
        case "+": result = left + right
        case "*": result = left * right
        case "^": result = left * right
    return result
```

Time

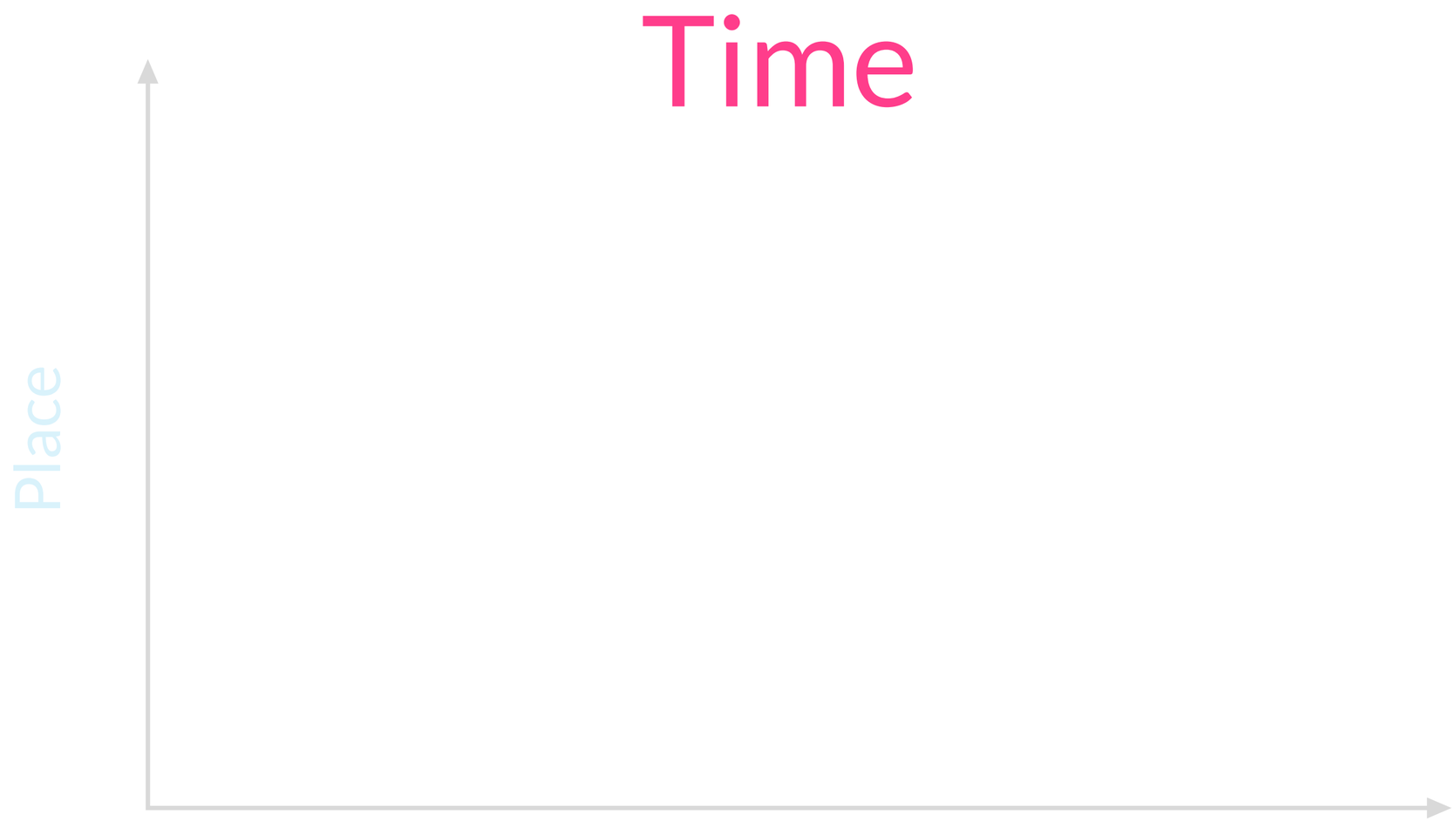
Place & Time


Place
node
node.value
node.left
node.right
left
right
node.op
result
Time

Place & Time



Place & Time



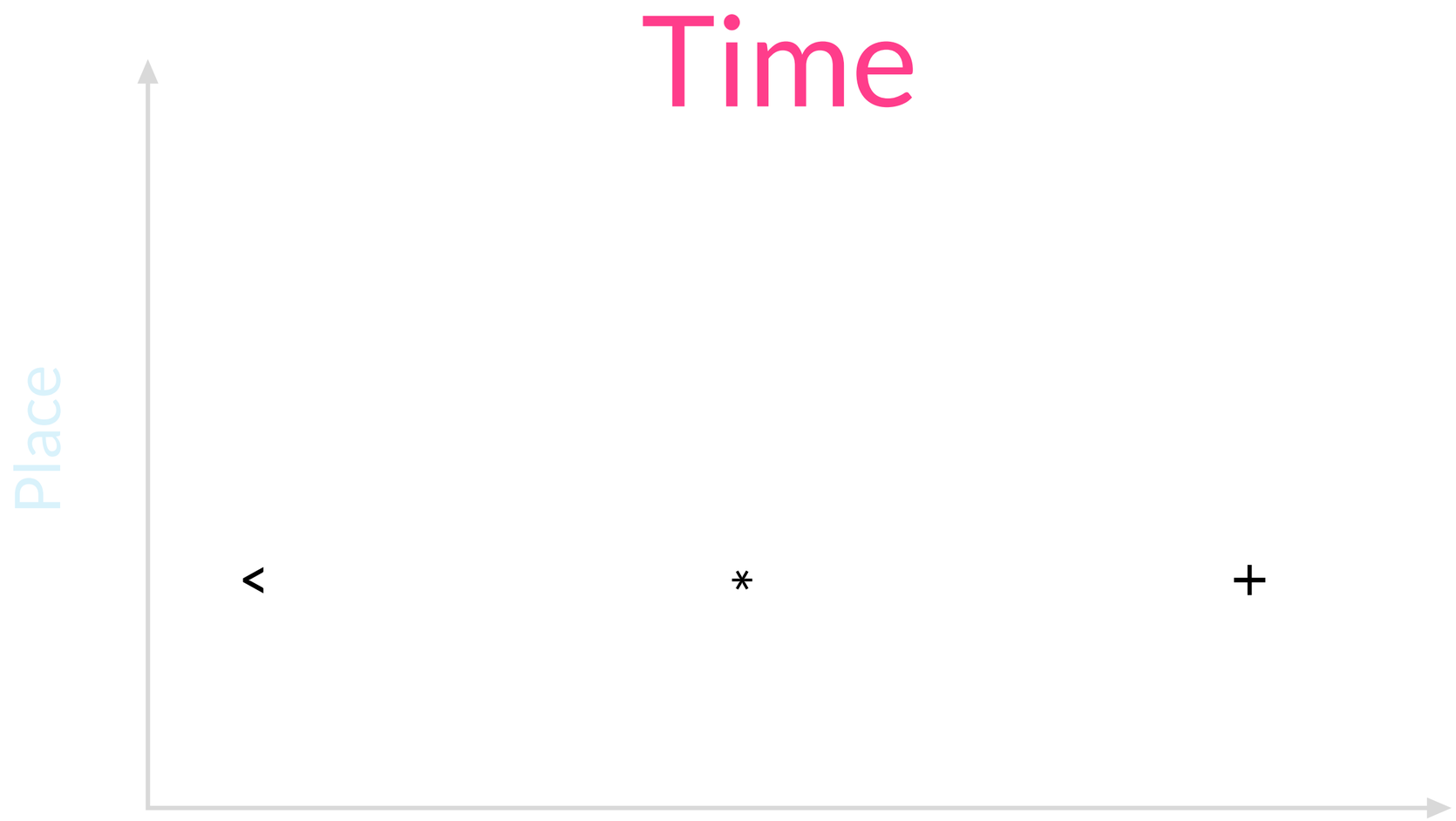
Place & Time

Time

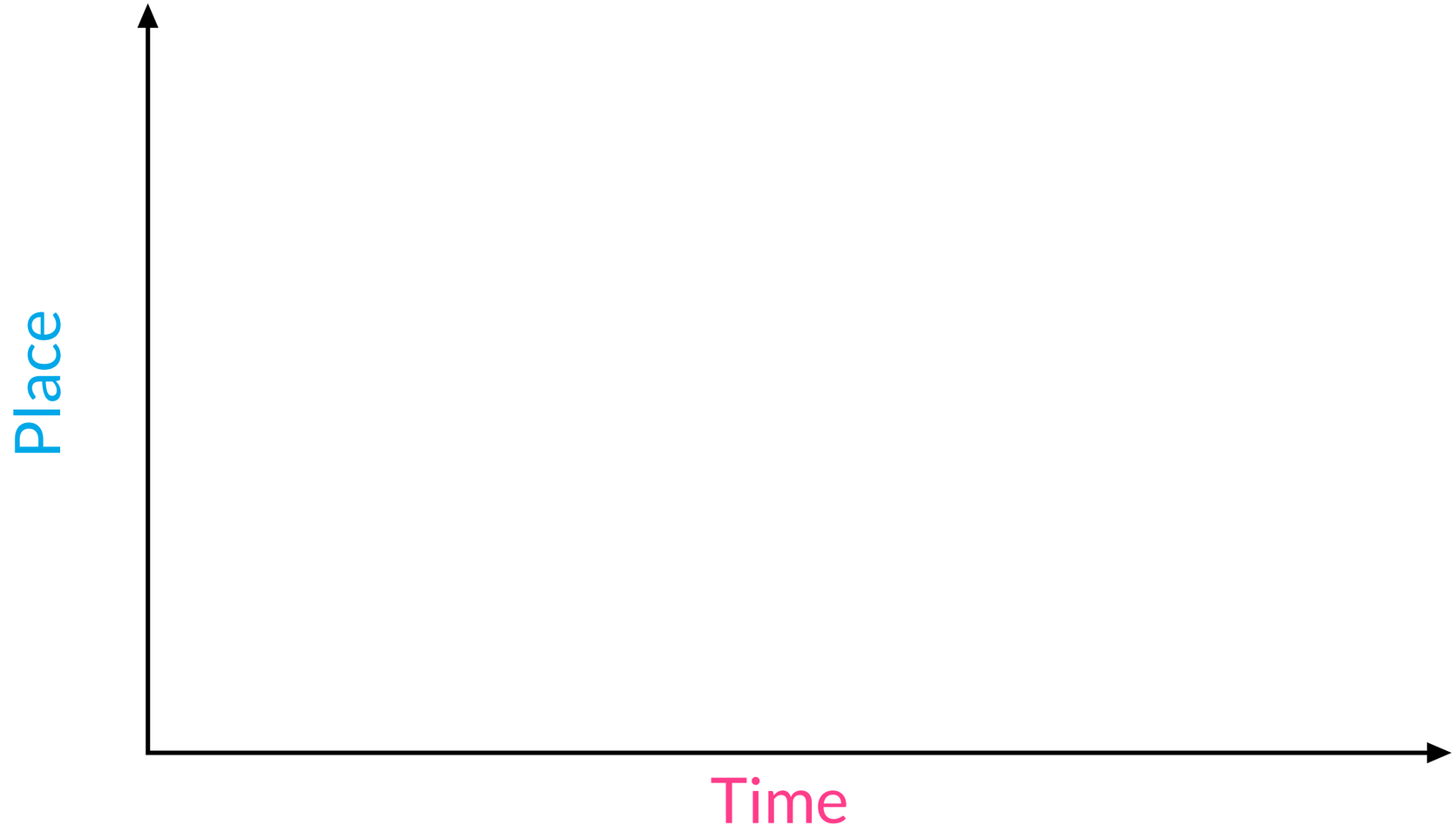
```
def evaluate(node):
    if isinstance(node, Num):
        return node.value
    left = evaluate(node.left)
    right = evaluate(node.right)
    match node.op:
        case "+": result = left + right
        case "*": result = left * right
        case "^": result = left * right
    print(node.op)
    return result
```

```
>>> calculate("2 + 3 * 4 ^ 2")
^
*
+
26
```

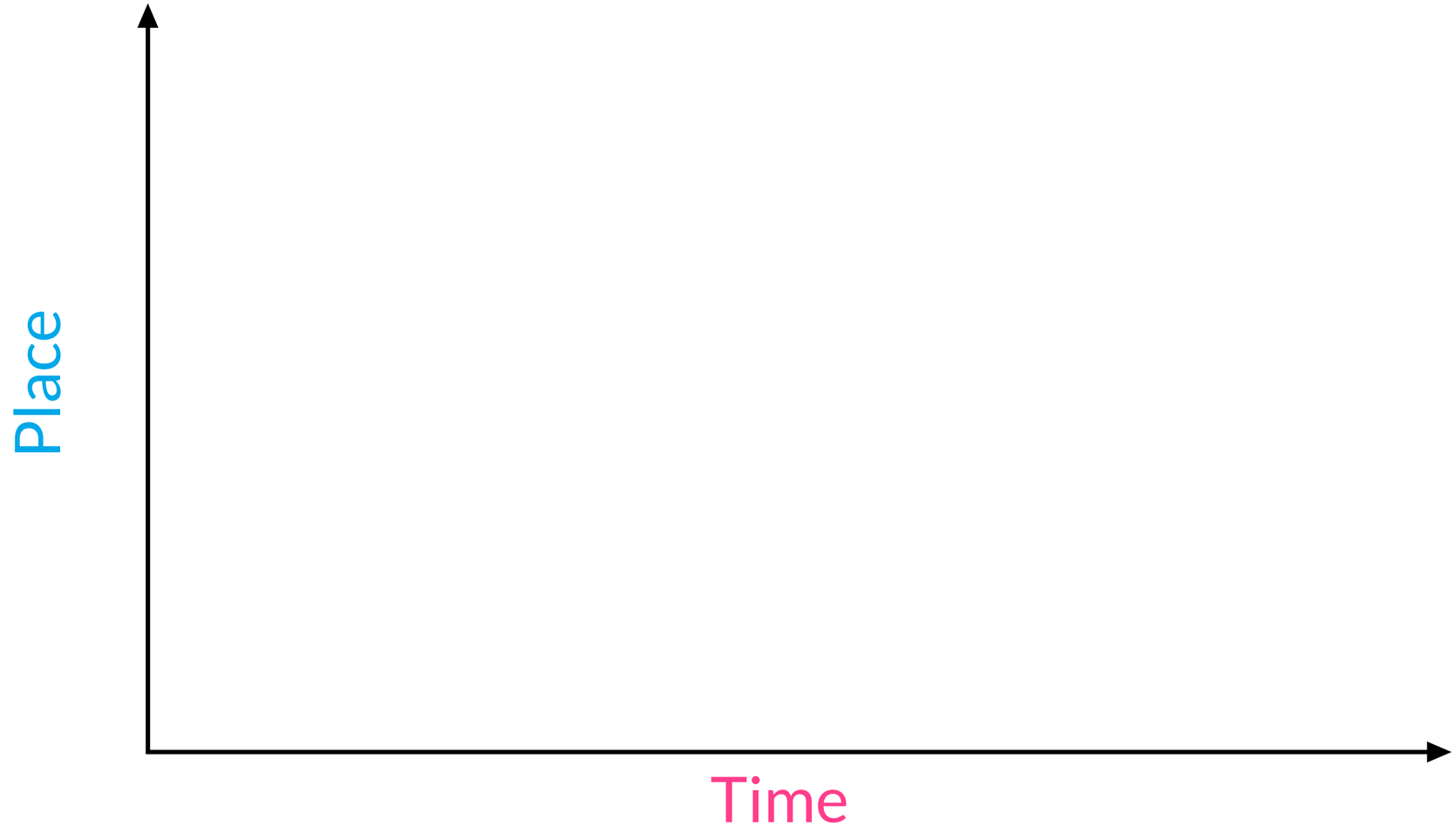
Place & Time



Place & Time



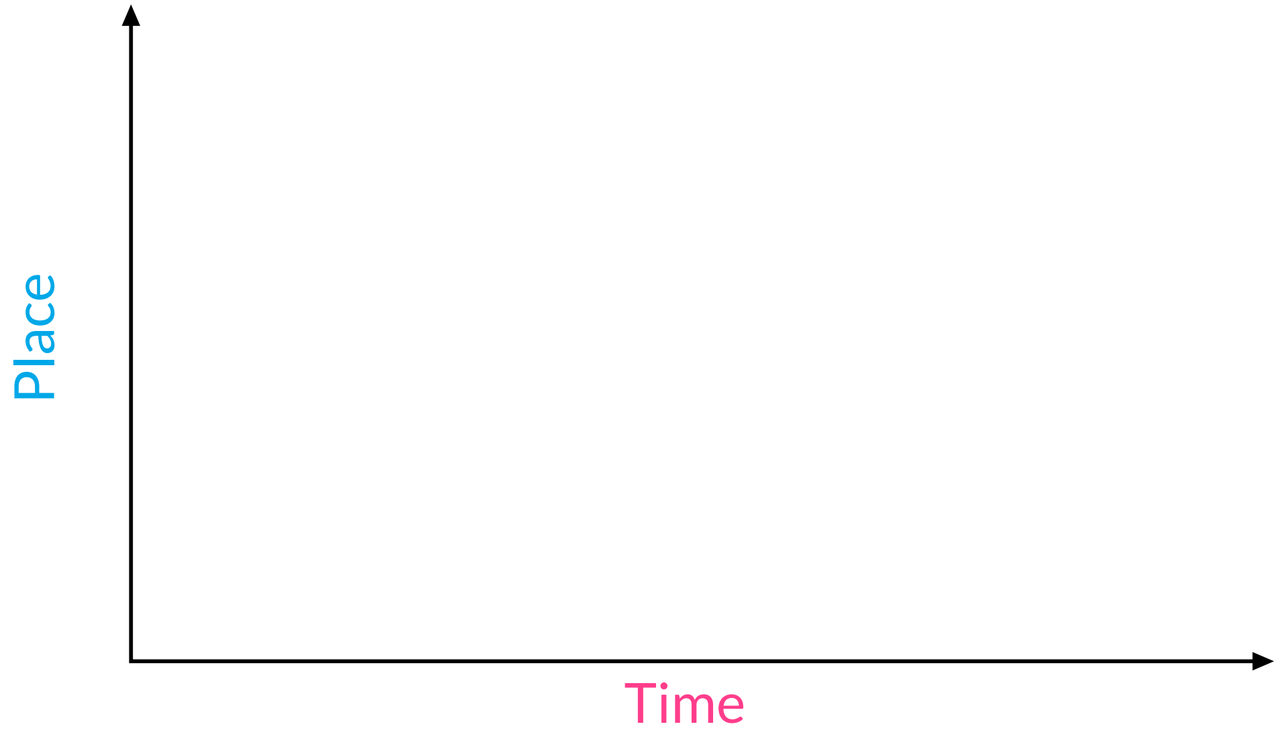
Place & Time



Place & Time

print debugging

Some Place · Across Time



evaluate(node)

```
1 def evaluate(node):
2     if isinstance(node, Num):
3         return node.value
4     left = evaluate(node.left)
5     right = evaluate(node.right)
6     match node.op:
7         case "+": result = left + right
8         case "*": result = left * right
9         case "^": result = left * right
10     return result
```

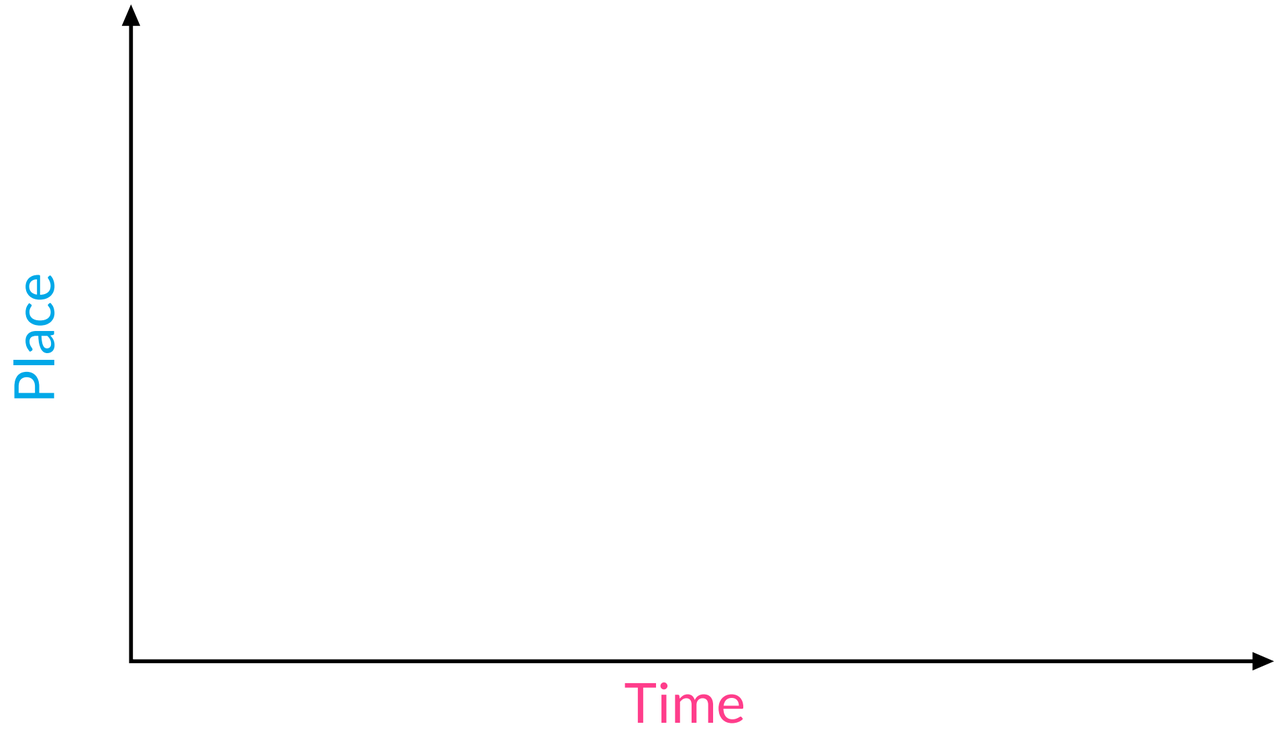
Terminal

```
>>> calculate("2 + 3 * 4 ^ 2")
```

Place & Time

print debugging

Some Place · Across Time



evaluate(node)

```
1 def evaluate(node):
2     if isinstance(node, Num):
3         return node.value
4     left = evaluate(node.left)
5     right = evaluate(node.right)
6     match node.op:
7         case "+": result = left + right
8         case "*": result = left * right
9         case "^": result = left * right
10     print(node.op)
11     return result
```

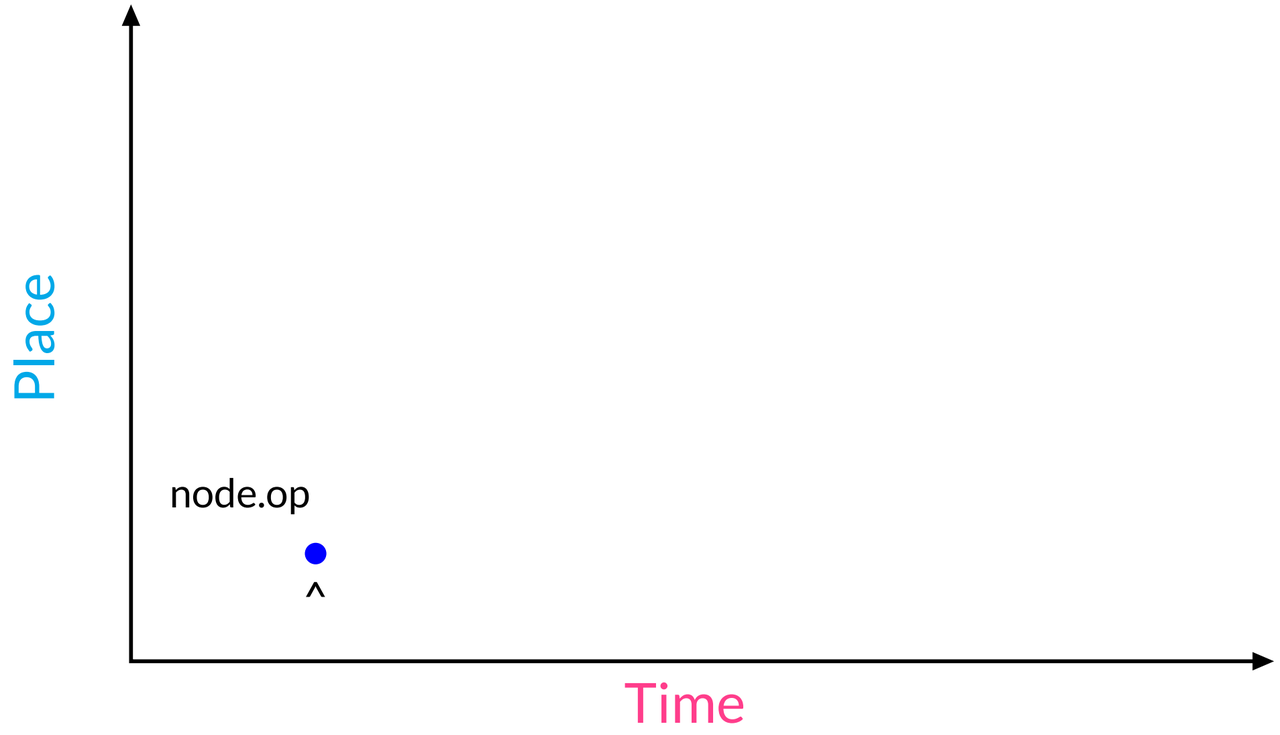
Terminal

```
>>> calculate("2 + 3 * 4 ^ 2")
```

Place & Time

print debugging

Some Place · Across Time




```
evaluate(node) · 4 ^ 2 → 8
1 def evaluate(node):
2     if isinstance(node, Num):
3         return node.value
4     left = evaluate(node.left)
5     right = evaluate(node.right)
6     match node.op:
7         case "+": result = left + right
8         case "*": result = left * right
9         case "^": result = left * right
10     print(node.op)
11     return result
```

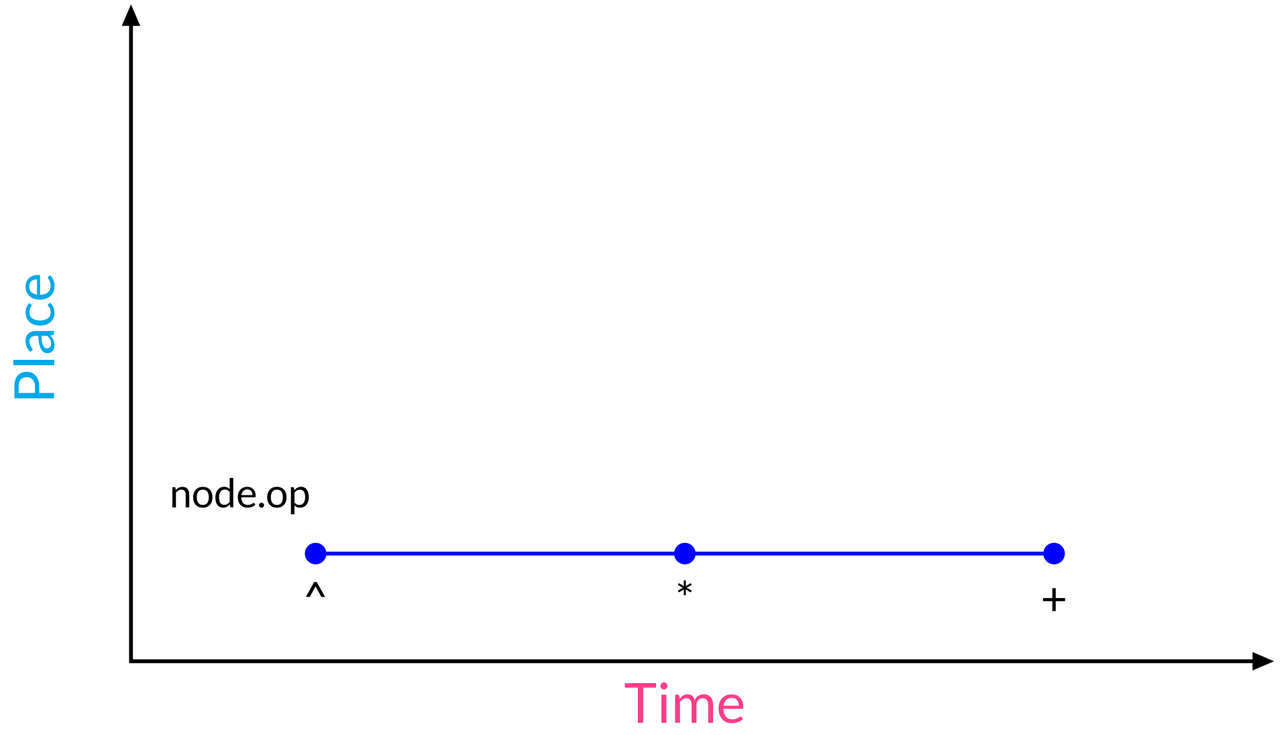
Terminal

```
>>> calculate("2 + 3 * 4 ^ 2")
^
```

Place & Time

print debugging

Some Place · Across Time



evaluate(node) · 2 + 24 → 26

```python
def evaluate(node):
    if isinstance(node, Num):
        return node.value
    left = evaluate(node.left)
    right = evaluate(node.right)
    match node.op:
        case "+": result = left + right
        case "*": result = left * right
        case "^": result = left * right
     print(node.op)
     return result
```

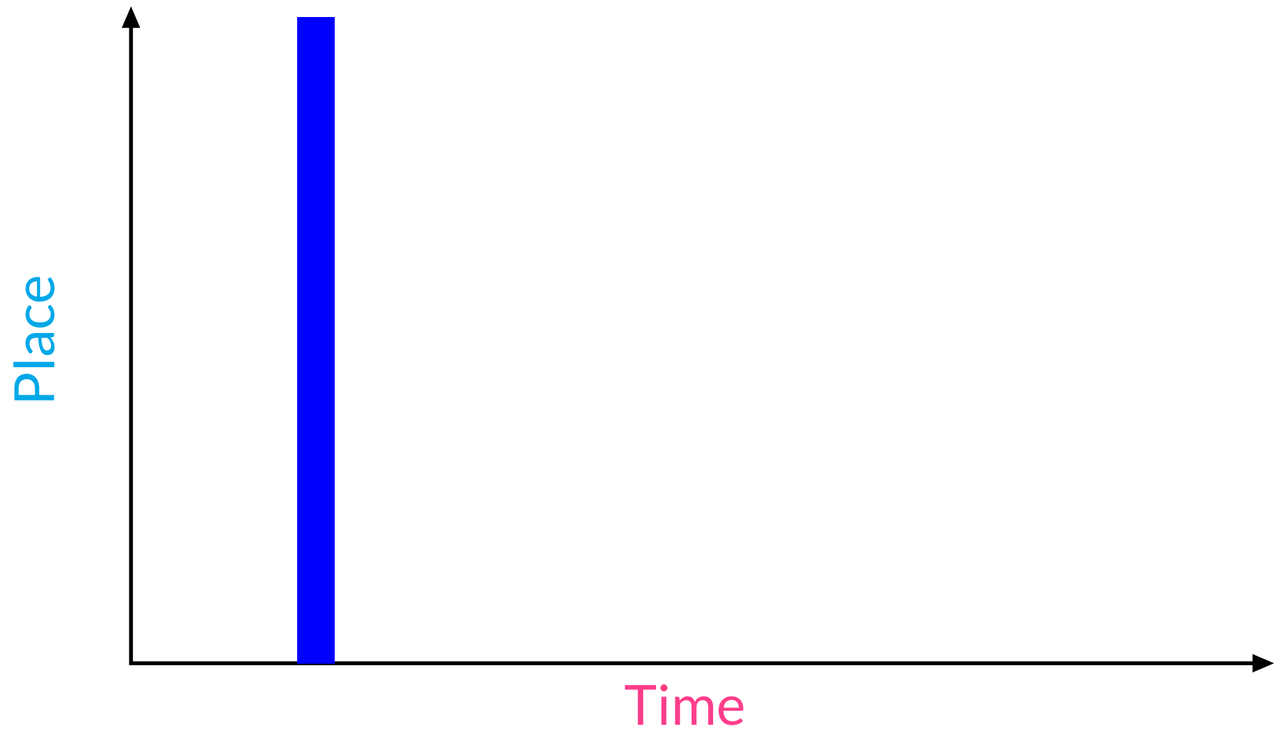
Terminal

```
>>> calculate("2 + 3 * 4 ^ 2")
^
*
+
26
```

Place & Time

breakpoint debugging

Across Place · Point in Time




```
evaluate(node) · 4 ^ 2
1 def evaluate(node):
2     if isinstance(node, Num):
3         return node.value
4     left = evaluate(node.left)
5     right = evaluate(node.right)
6     match node.op:
7         case "+": result = left + right
8         case "*": result = left * right
9         case "^": result = left * right
10     return result
```

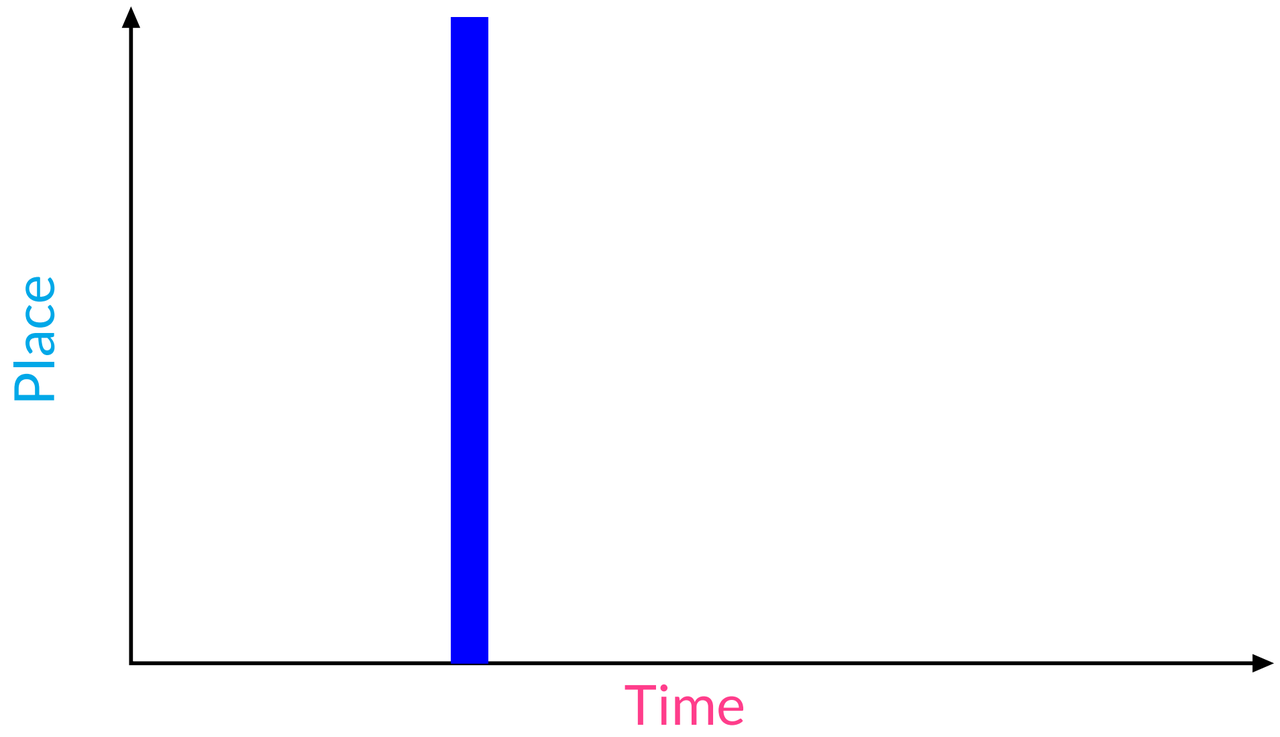
Variables · evaluate

node	BinOp("^")	node.value	–
node.left	Num(4)	node.right	Num(2)
left	4	right	2
node.op	"^"	result	not assigned

Place & Time

breakpoint debugging

Across Place · Point in Time

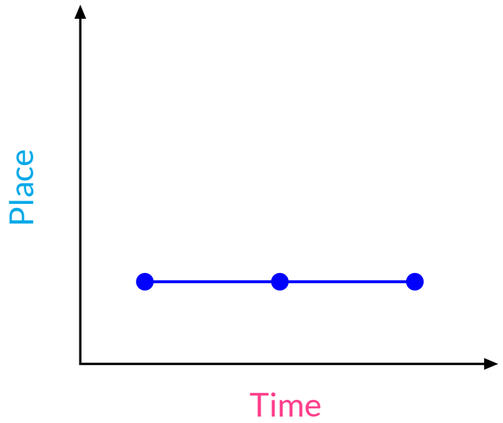



```
evaluate(node) · 4 ^ 2
1 def evaluate(node):
2     if isinstance(node, Num):
3         return node.value
4     left = evaluate(node.left)
5     right = evaluate(node.right)
6     match node.op:
7         case "+": result = left + right
8         case "*": result = left * right
9         case "^": result = left * right
10     return result
```

Variables · evaluate

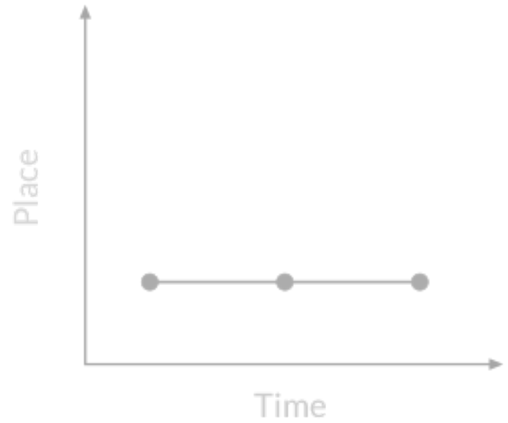
node	BinOp("^")	node.value	–
node.left	Num(4)	node.right	Num(2)
left	4	right	2
node.op	"^"	result	8

Debuggers Under The Lens

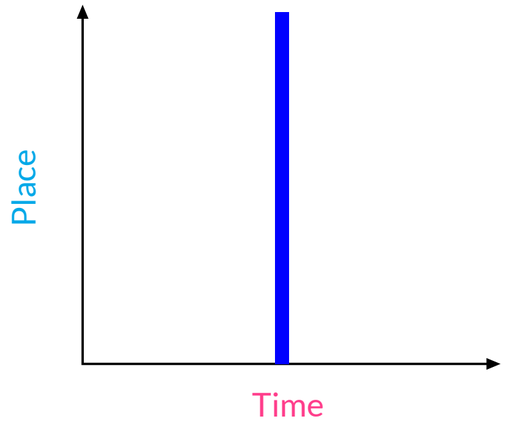


Print debugging

Debuggers Under The Lens

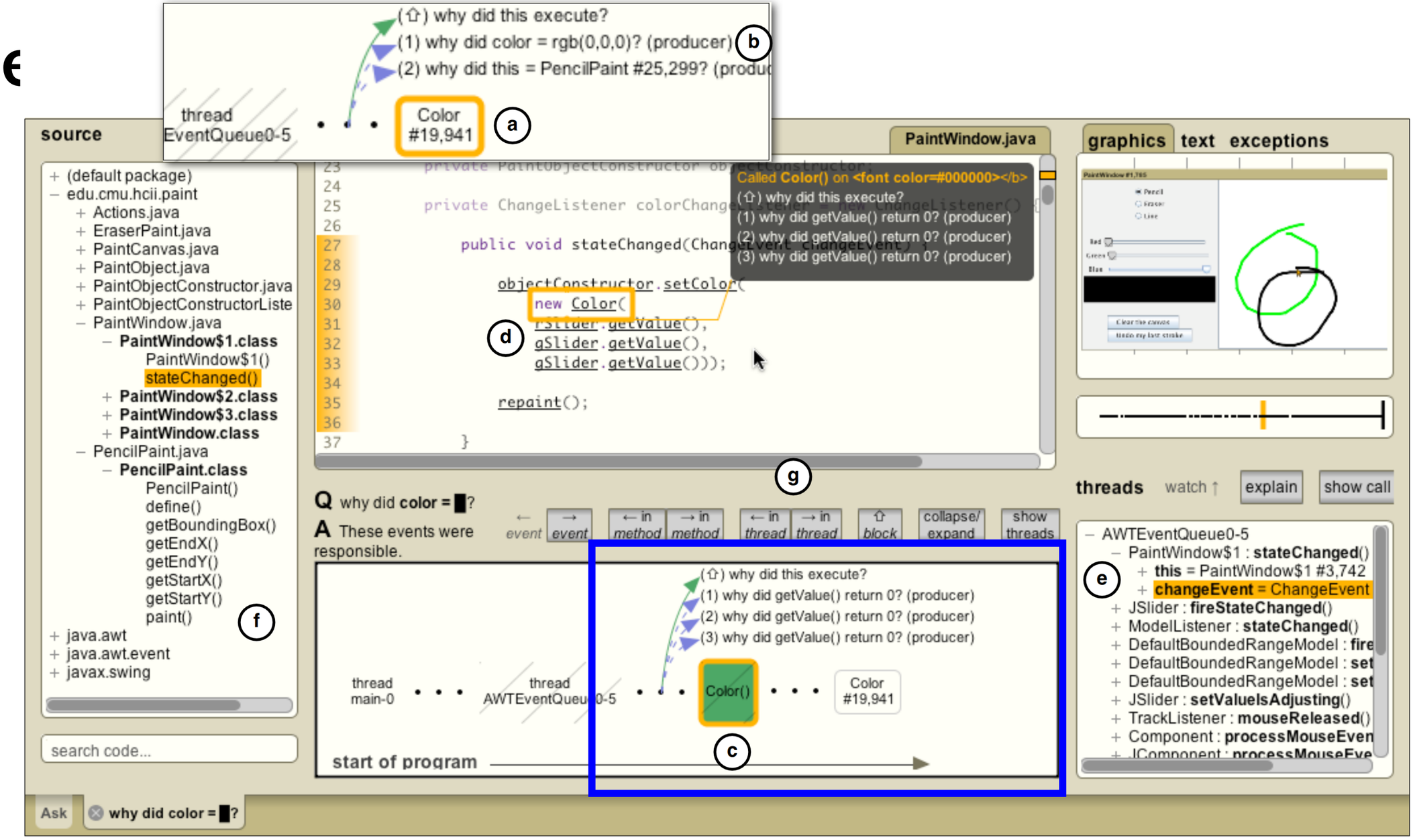


Print debugging

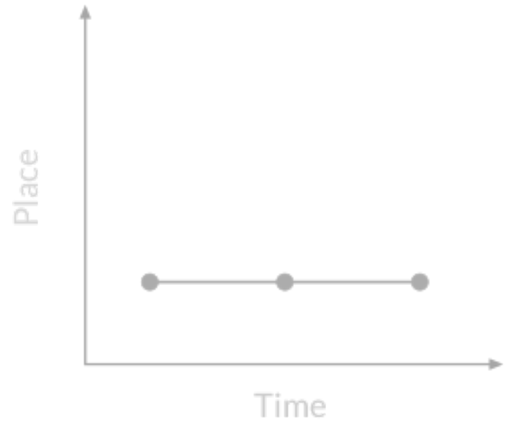


Breakpoint debugging

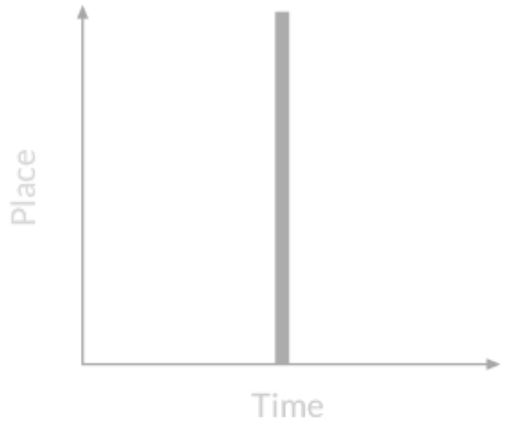
De


(⇧) why did this execute?
(1) why did color = rgb(0,0,0)? (producer)
(2) why did this = PencilPaint #25,299? (produc
b
thread
EventQueue0-5
Color
#19,941
a
source
PaintWindow.java
graphics
text
exceptions
+ (default package)
– edu.cmu.hcii.paint
+ Actions.java
+ EraserPaint.java
+ PaintCanvas.java
+ PaintObject.java
+ PaintObjectConstructor.java
+ PaintObjectConstructorListe
– PaintWindow.java
– PaintWindow$1.class
PaintWindow$1()
stateChanged()
+ PaintWindow$2.class
+ PaintWindow$3.class
+ PaintWindow.class
– PencilPaint.java
– PencilPaint.class
PencilPaint()
define()
getBoundingBox()
getEndX()
getEndY()
getStartX()
getStartY()
paint()
+ java.awt
+ java.awt.event
+ javax.swing
f
search code...
Called Color() on <font color=#000000>
(⇧) why did this execute?
(1) why did getValue() return 0? (producer)
(2) why did getValue() return 0? (producer)
(3) why did getValue() return 0? (producer)
objectConstructor.setColor(
new Color(
gSlider.getValue(),
gSlider.getValue(),
gSlider.getValue()));
repaint();
d
g
Q why did color = ?
A These events were responsible.
← event
→ event
← in method
→ in method
← in thread
→ in thread
⇧ block
collapse/ expand
show threads
thread
main-0
thread
AWTEventQueue-0-5
Color()
Color
#19,941
c
start of program
threads
watch ↑
explain
show call
– AWTEventQueue0-5
– PaintWindow$1 : stateChanged()
+ this = PaintWindow$1 #3,742
+ changeEvent = ChangeEvent
+ JSlider : fireStateChanged()
+ ModelListener : stateChanged()
+ DefaultBoundedRangeModel : fire
+ DefaultBoundedRangeModel : set
+ DefaultBoundedRangeModel : set
+ JSlider : setValueIsAdjusting()
+ TrackListener : mouseReleased()
+ Component : processMouseEven
e
Ask
why did color = ?

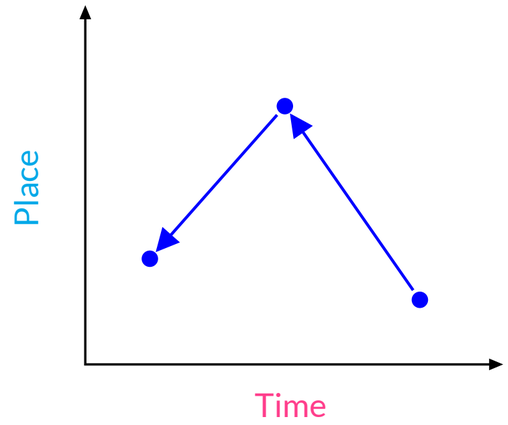
Debuggers Under The Lens



Print debugging

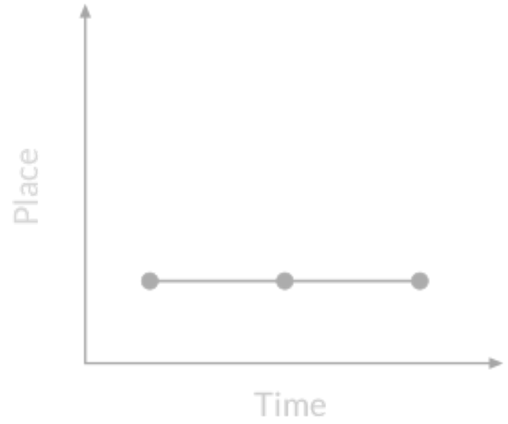


Breakpoint debugging

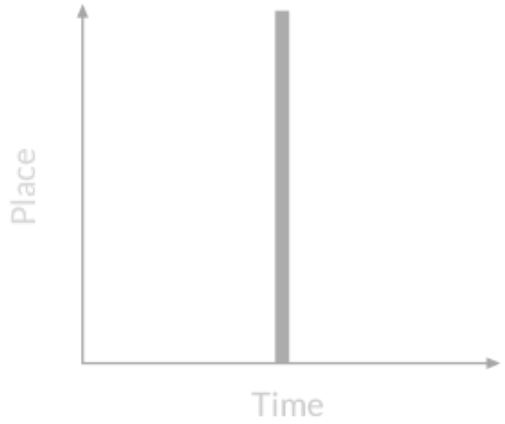


Whyline

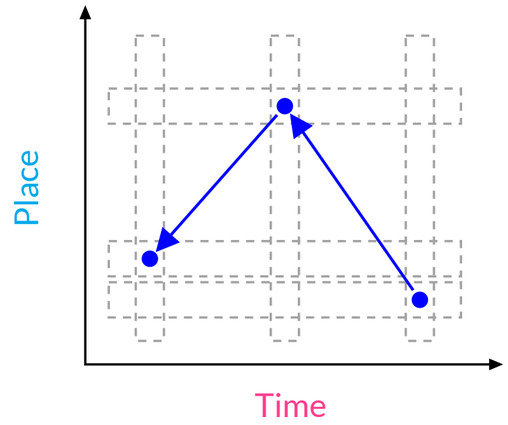
Debuggers Under The Lens



Print debugging

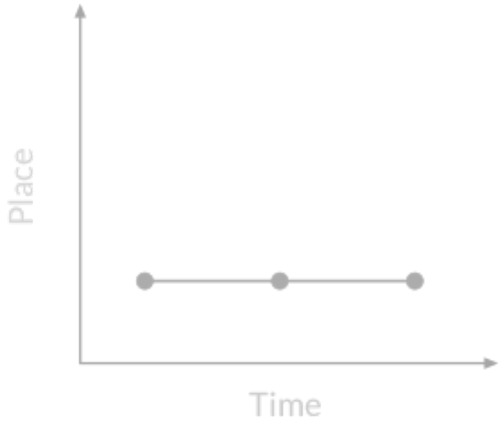


Breakpoint debugging

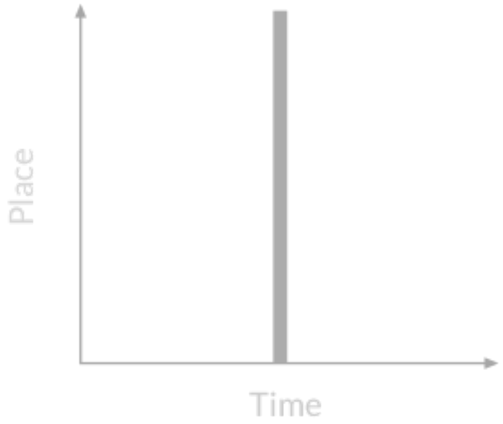


Whyline

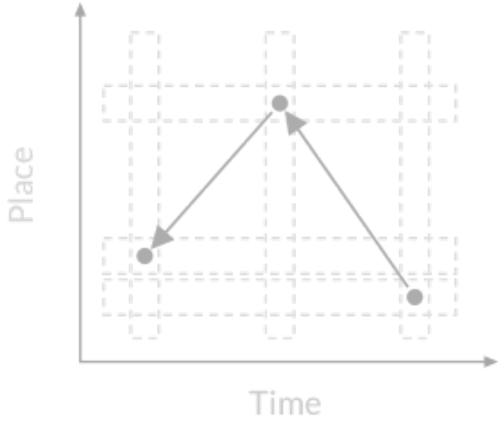
Debuggers Under The Lens



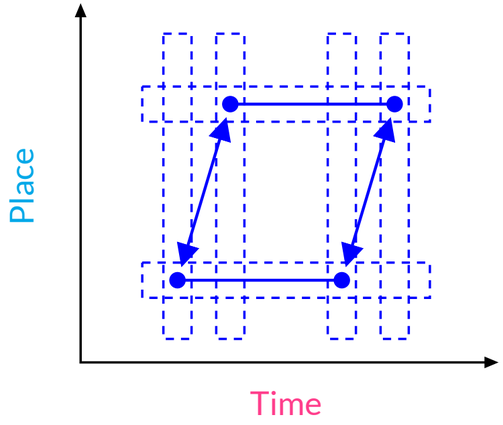
Print debugging



Breakpoint debugging



Whyline



Future Tools

Study Findings

Formative Interviews, Comparative Tool-Use Lab Study

Formative Interviews

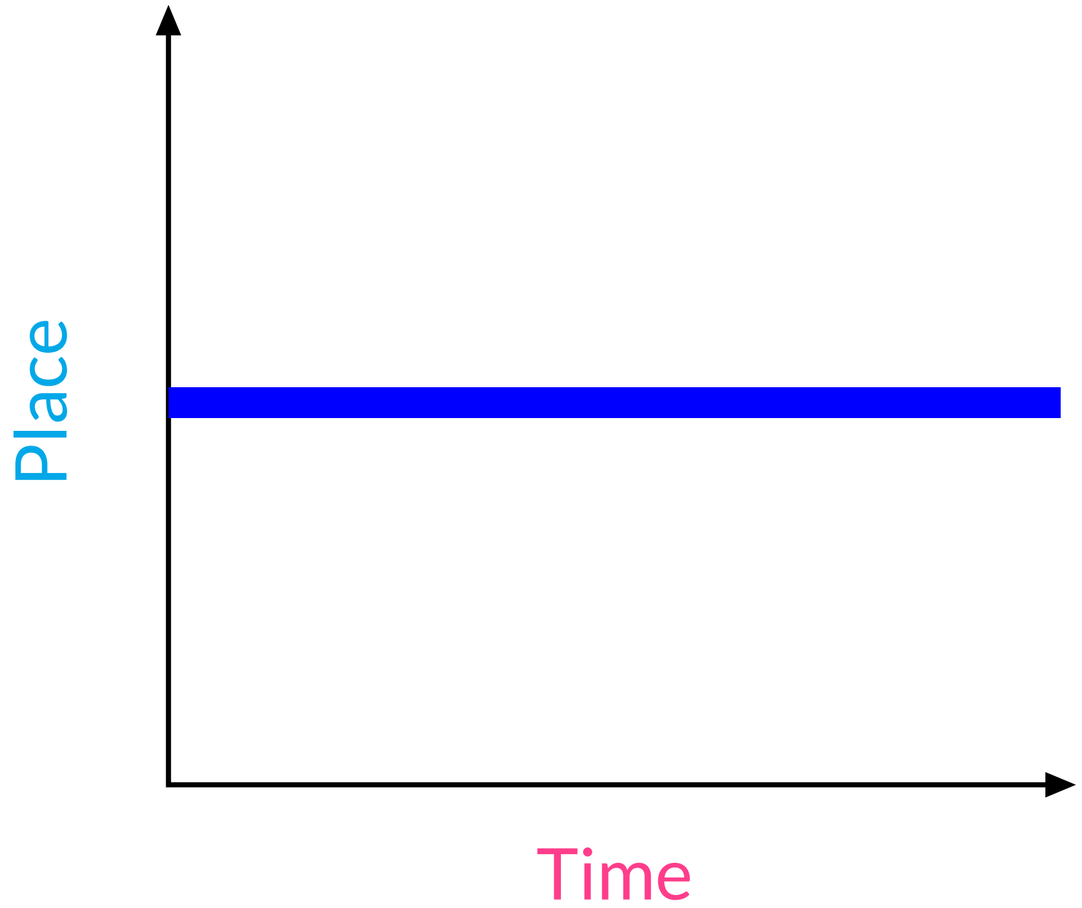
12 industry software engineers, *"How do you debug the systems you work on?"*

Comparative Tool-Use Lab Study

19 undergrad + grad students, 2 debugging tasks, 3 debugging tools

Types of Queries

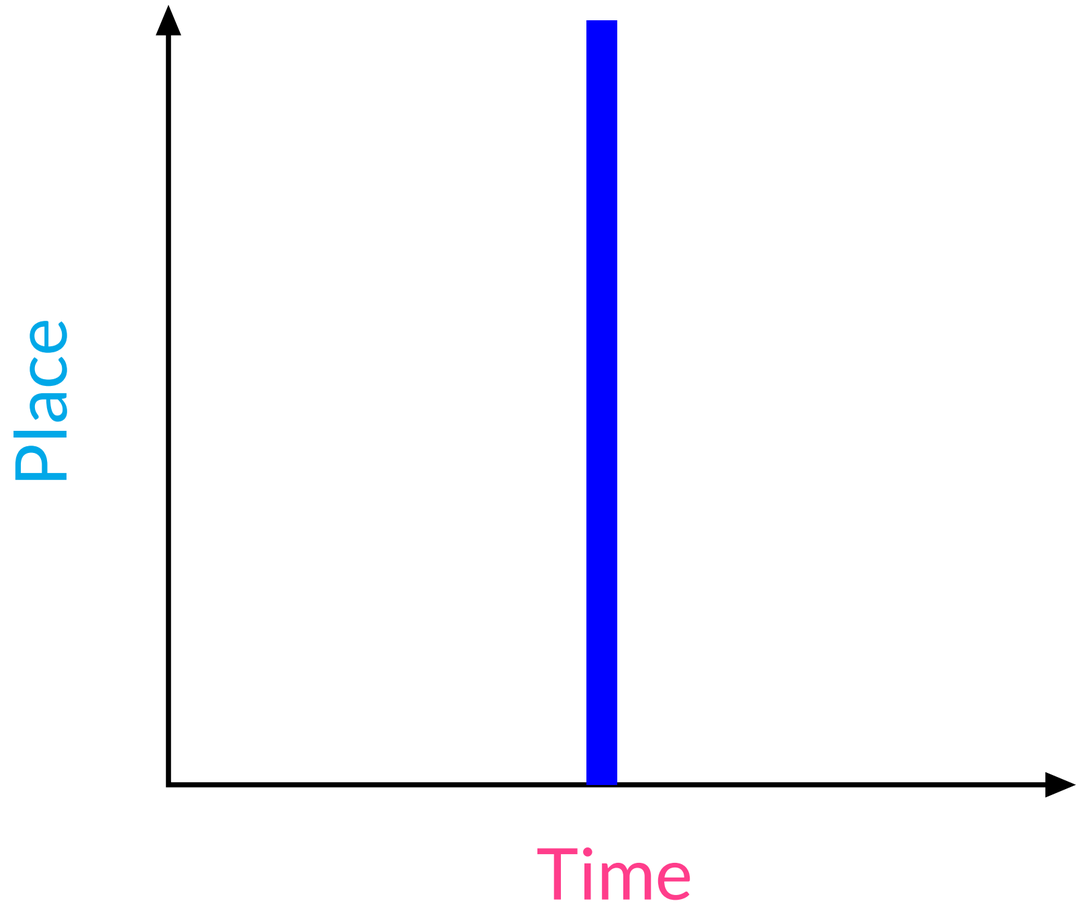
Across Time (17 participants, 58 instances)



"What are all of the values this can have?"

Types of Queries

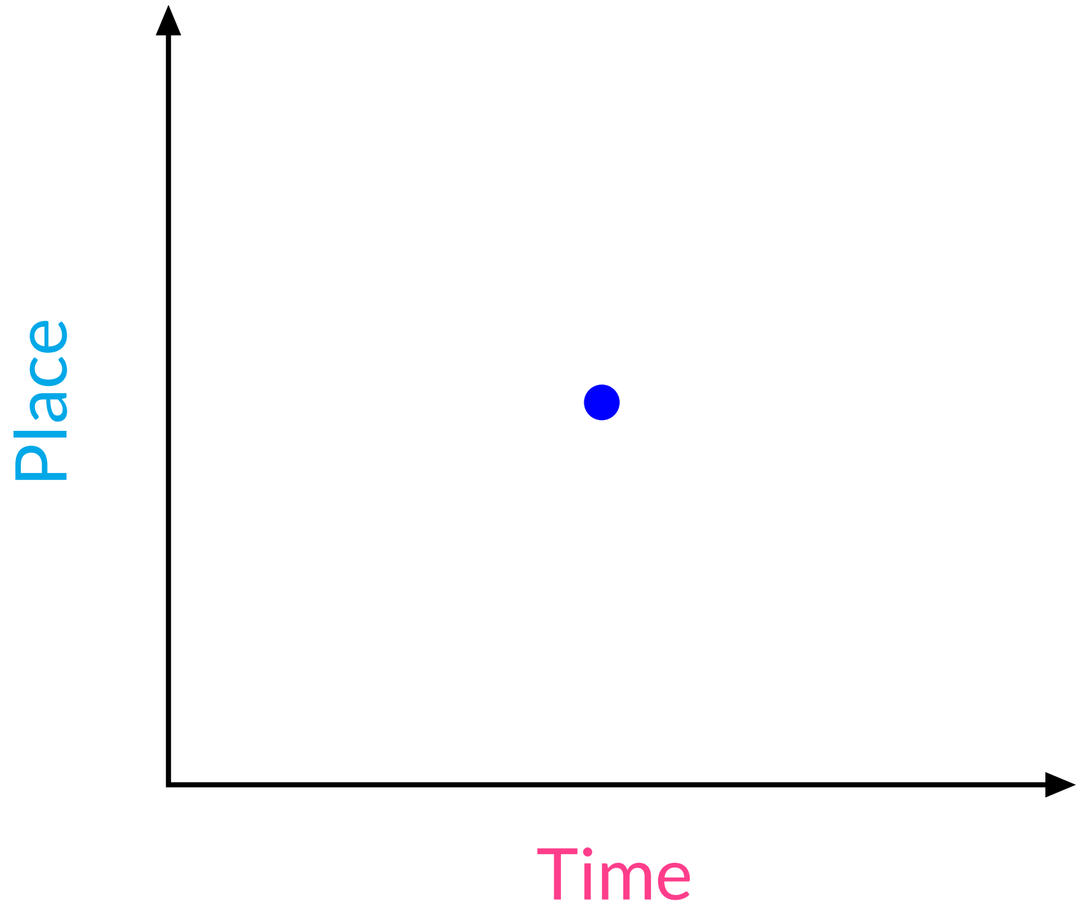
Across Place (6 participants, 9 instances)



"What else is here?"

Types of Queries

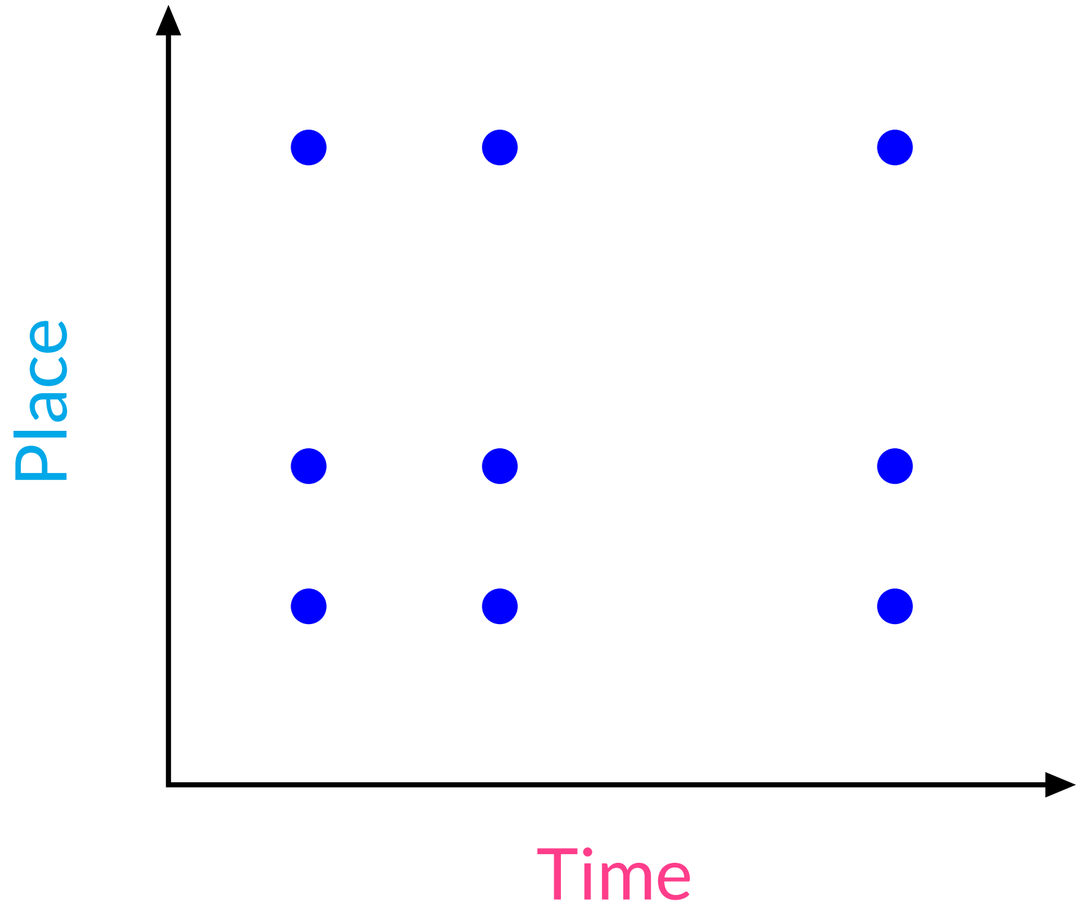
Point in Place and Time (7 participants, 12 instances)



"Did this ever happen?"

Types of Queries

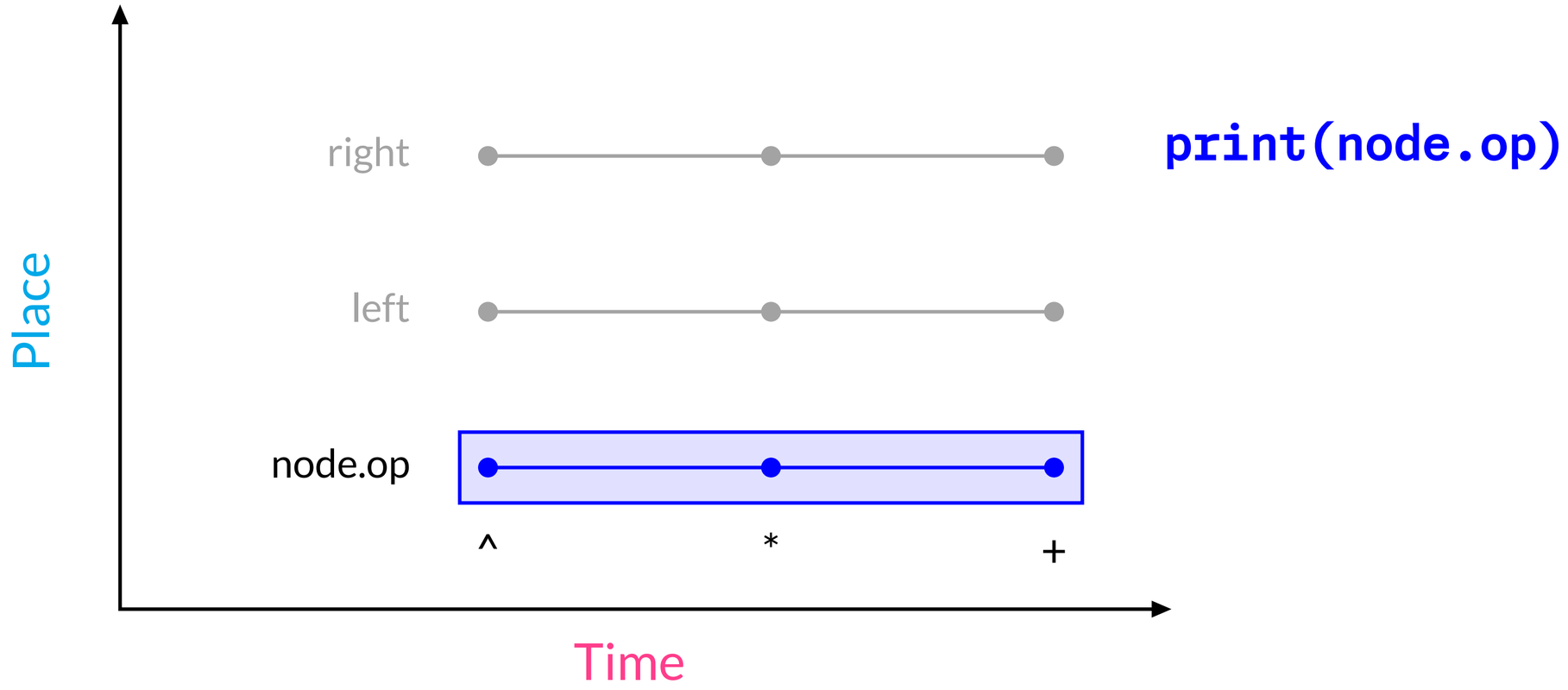
Coordinating Across Place and Time (7 participants)



`log(middleware, response)`

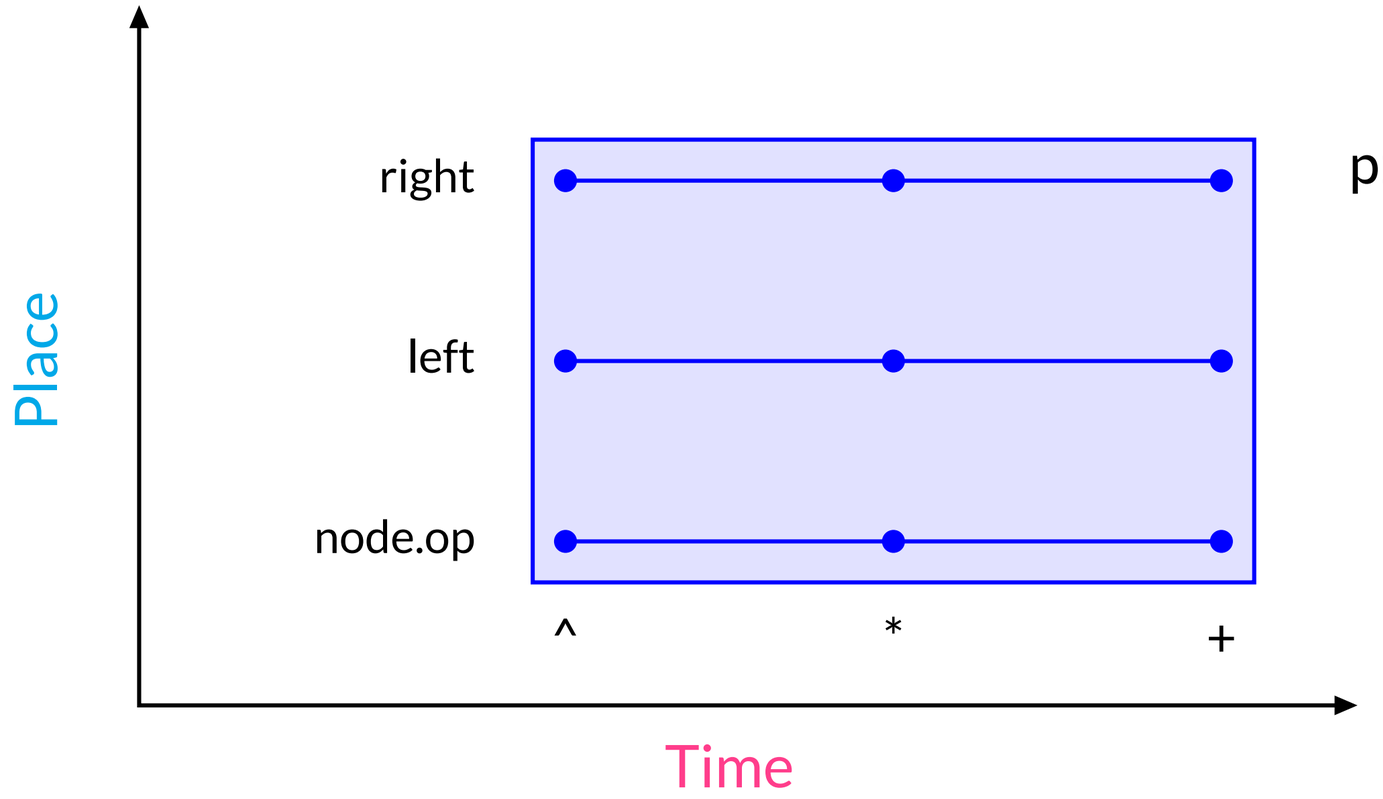
"When the request is rejected, which middleware was running? Oh, a different number of middleware run per-request and multiple different middleware cause rejections."

Transformations



Transformations

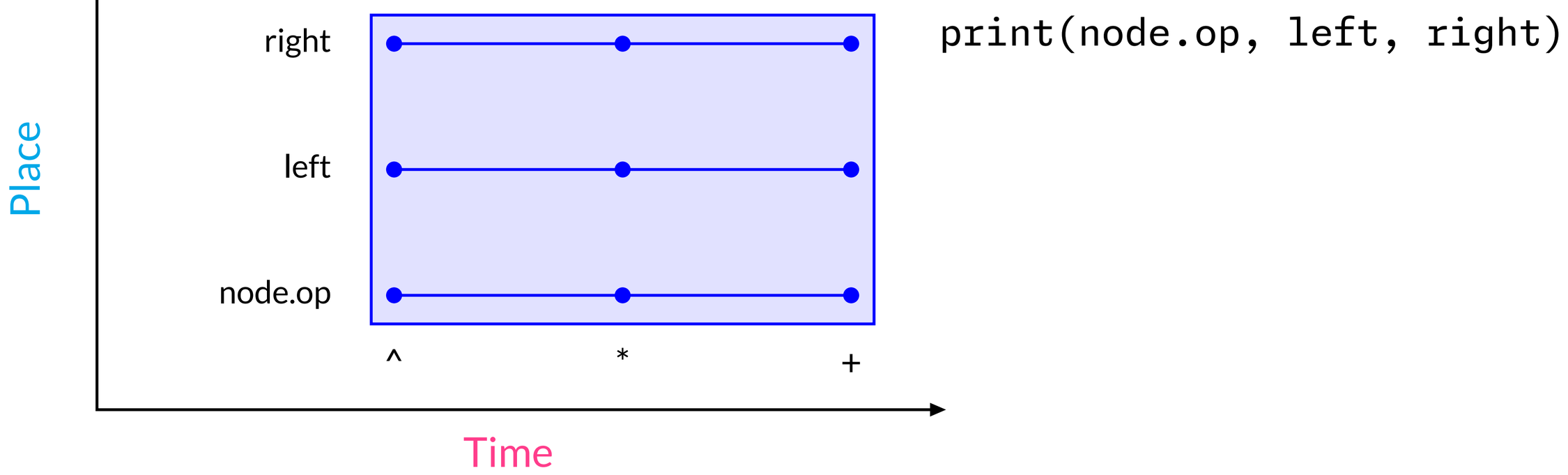
Widen across place (14 participants, 27 instances)




```
print(node.op, left, right)
```

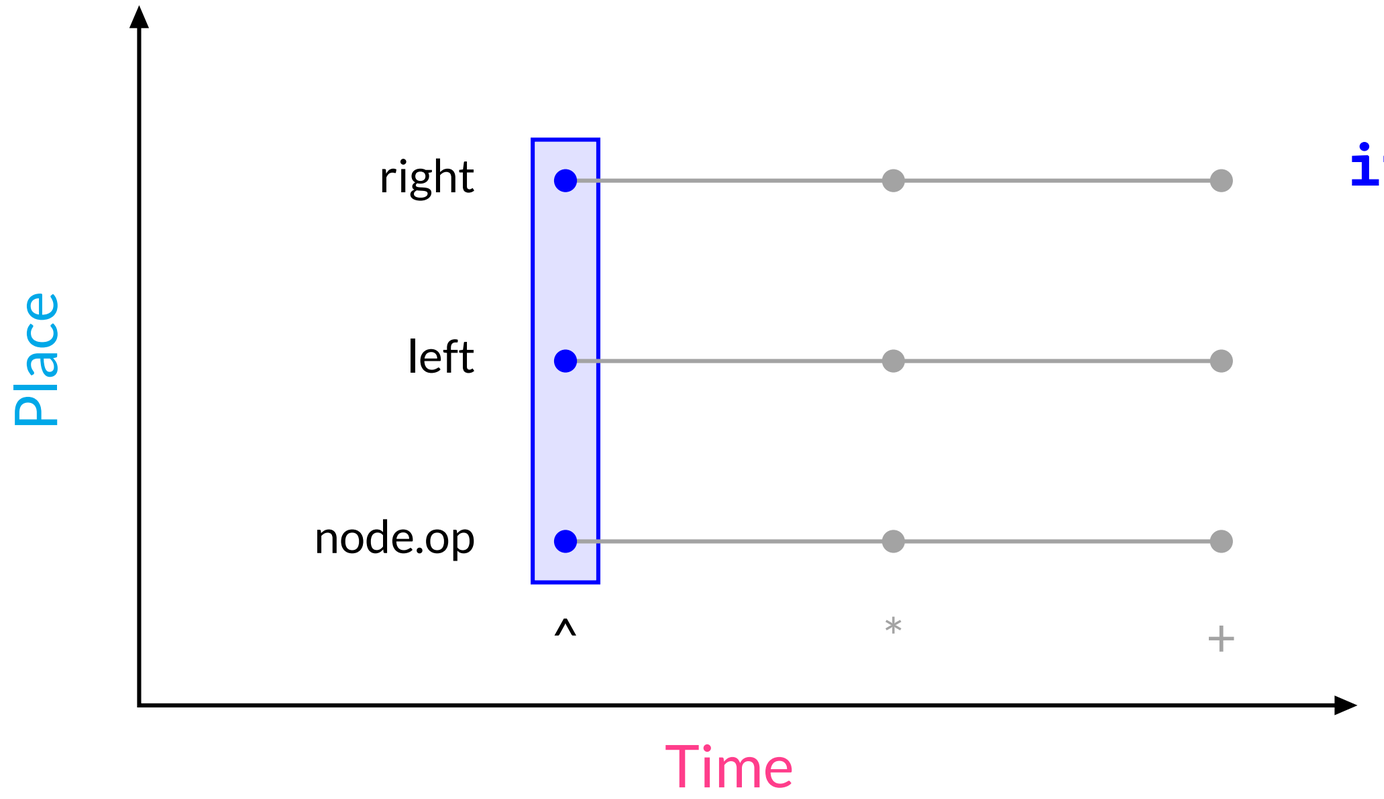
Transformations

Takeaway:
Do you support transformations? Via what interaction?



Transformations

Narrow in time (12 participants, 18 instances)




```
if node.op == "^":
    print(node.op, left, right)
```

Pre-Filtering

Pre-Filtering

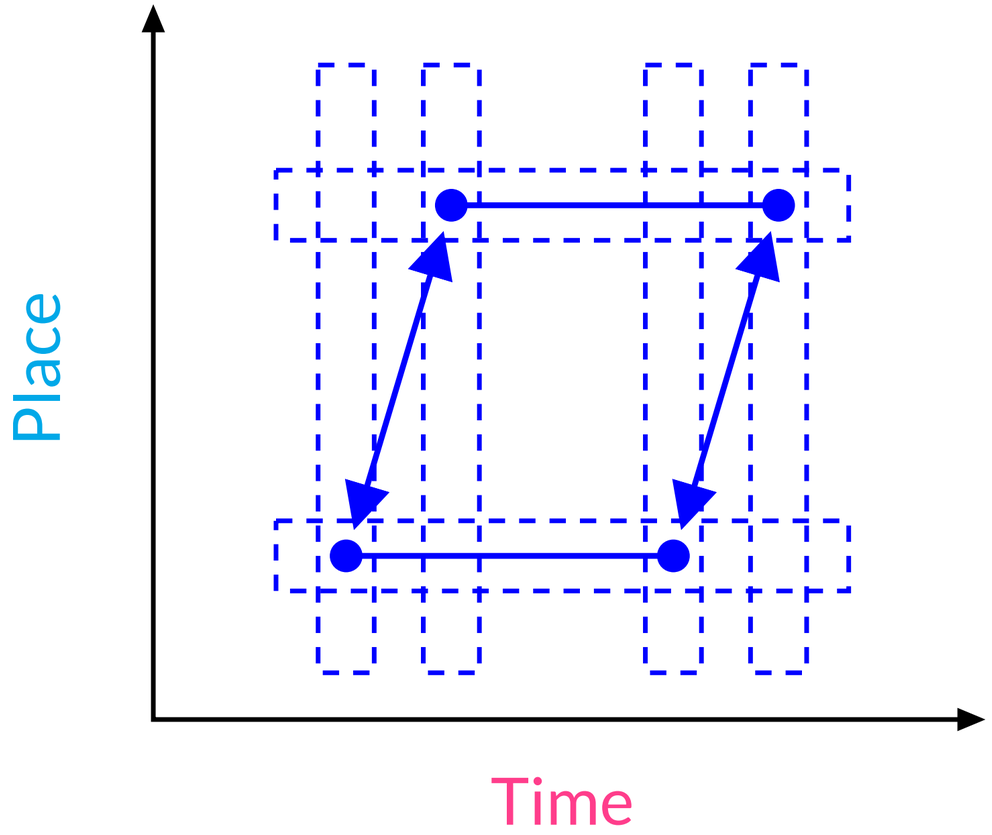
Programmers cope with tool limitations by asking for less

Pre-Filtering

Programmers cope with tool limitations by asking for less

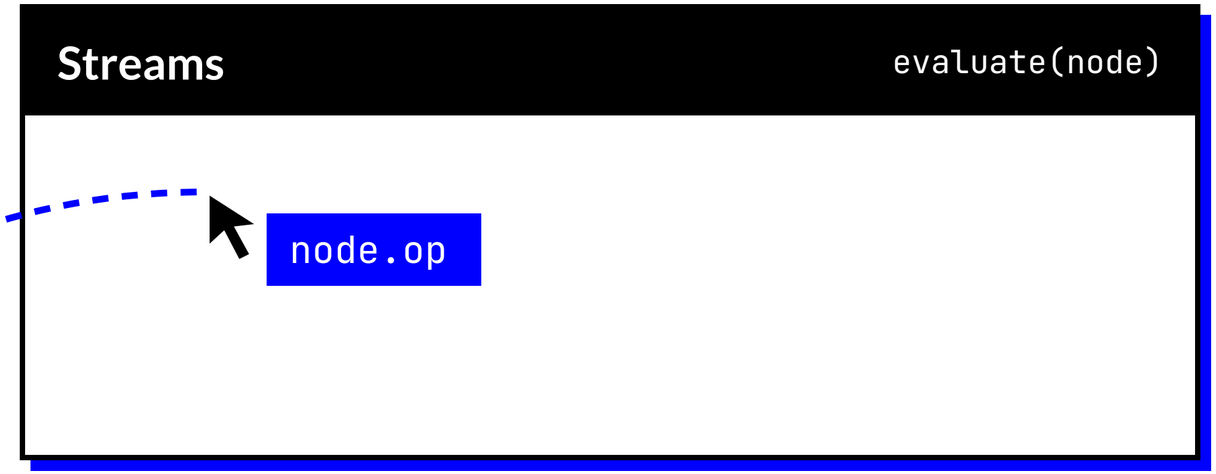
"there will be a lot of statements, right? ...it looks like too much information for me to look at and process and get an output"

The Future


Place
Time

```python
def evaluate(node):
    if isinstance(node, Num):
        return node.value
    left = evaluate(node.left)
    right = evaluate(node.right)
    match node.op:
        case "+": result = left + right
        case "*": result = left * right
        case "^": result = left * right
     return result
```

```python
def evaluate(node):
    if isinstance(node, Num):
        return node.value
    left = evaluate(node.left)
    right = evaluate(node.right)
    match node.op:
        case "+": result = left + right
        case "*": result = left * right
        case "^": result = left * right
     return result
```

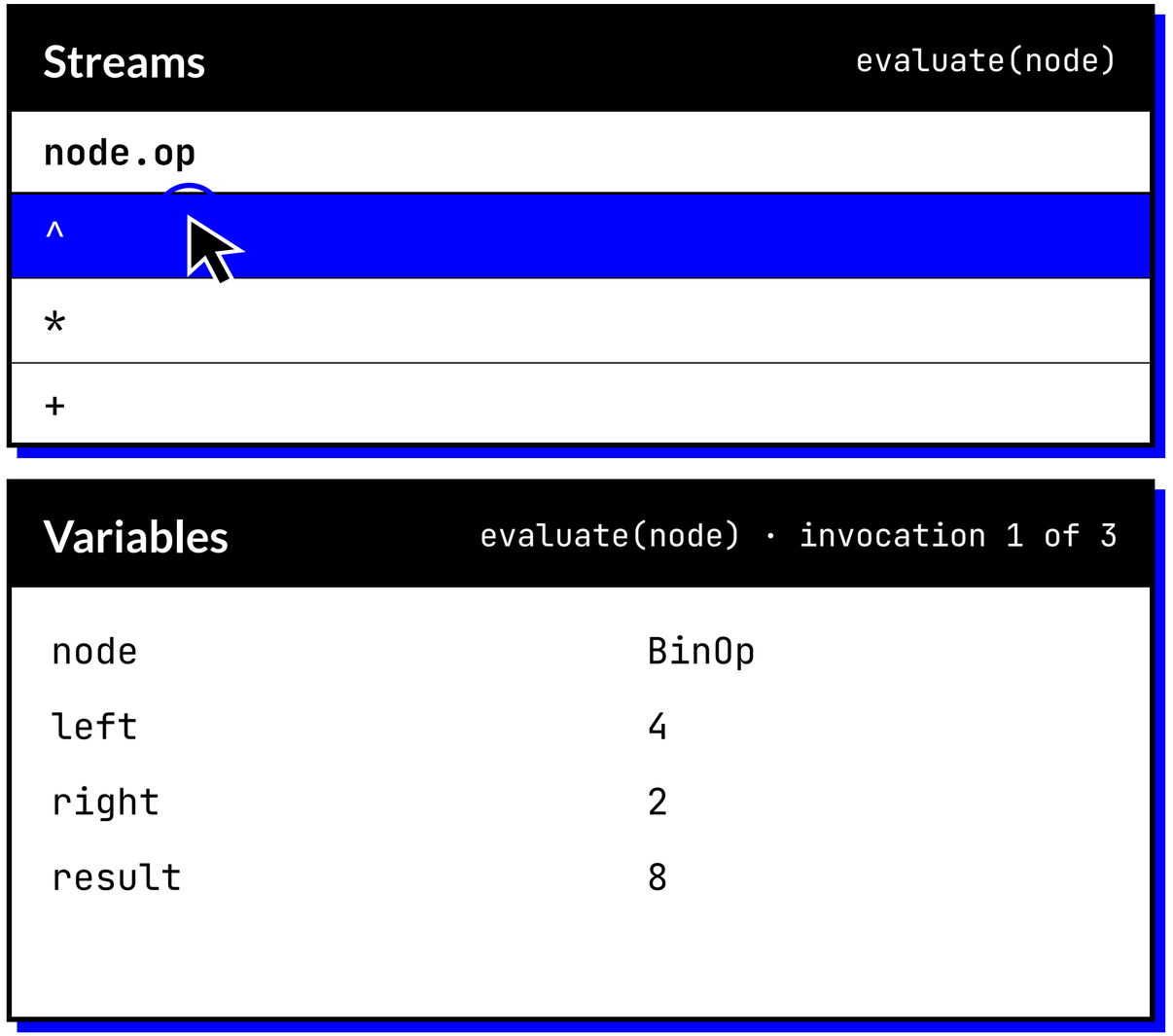


```python
def evaluate(node):
    if isinstance(node, Num):
        return node.value
    left = evaluate(node.left)
    right = evaluate(node.right)
    match node.op:
        case "+": result = left + right
        case "*": result = left * right
        case "^": result = left * right
     return result
```

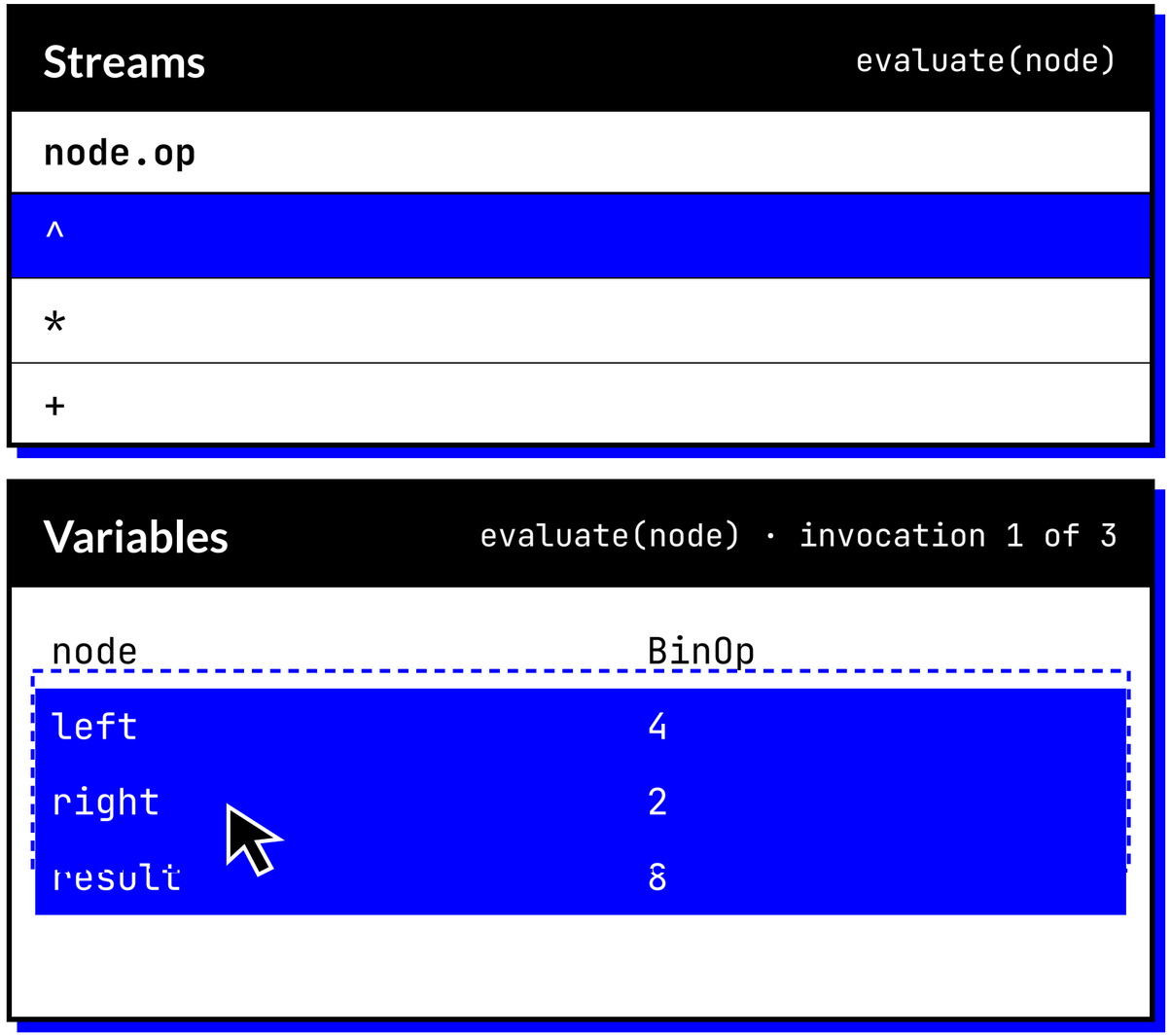
Streams `evaluate(node)`

node.op
^
*
+

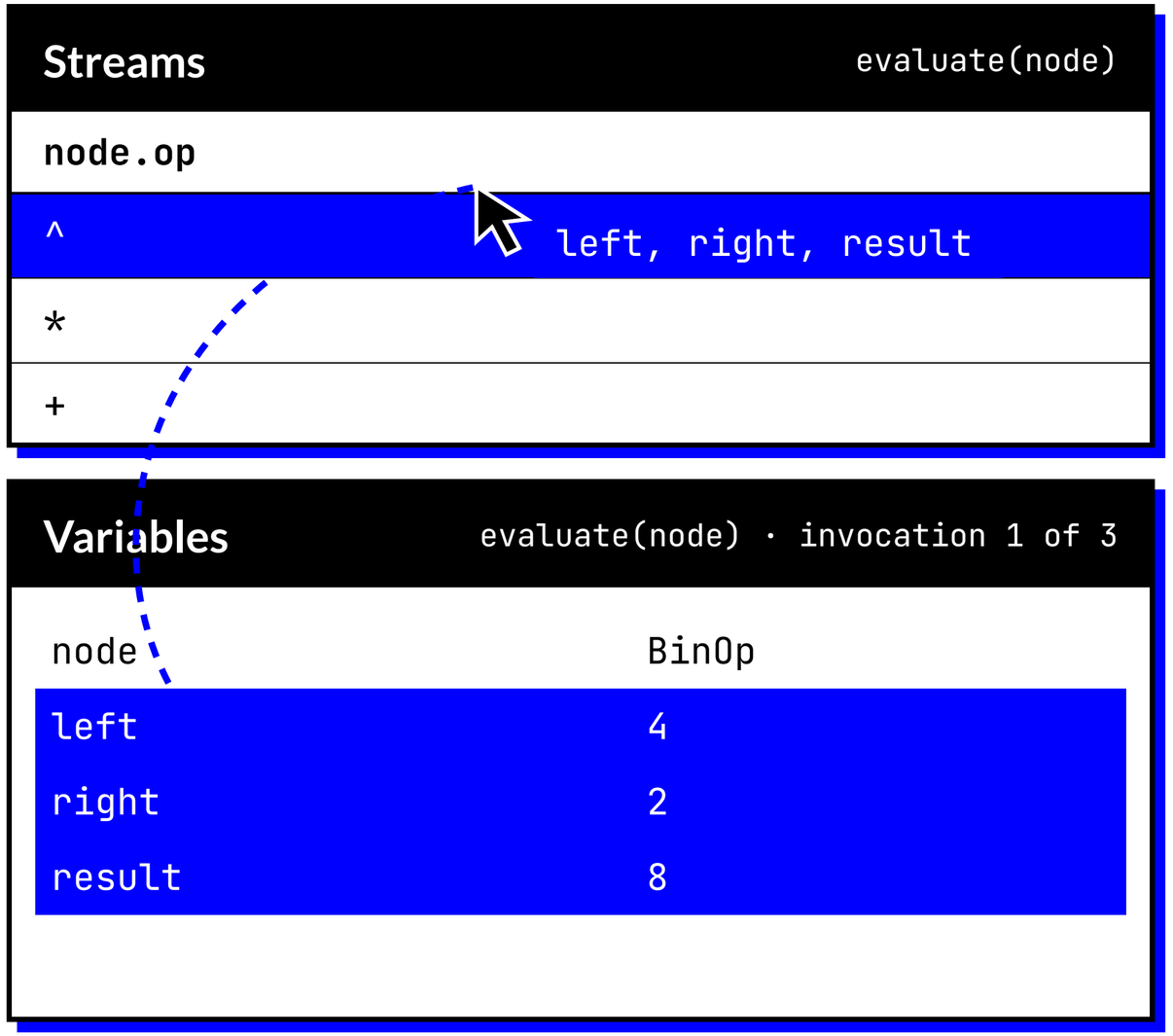
```python
def evaluate(node):
    if isinstance(node, Num):
        return node.value
    left = evaluate(node.left)
    right = evaluate(node.right)
    match node.op:
        case "+": result = left + right
        case "*": result = left * right
        case "^": result = left * right
     return result
```



```python
def evaluate(node):
    if isinstance(node, Num):
        return node.value
    left = evaluate(node.left)
    right = evaluate(node.right)
    match node.op:
        case "+": result = left + right
        case "*": result = left * right
        case "^": result = left * right
     return result
```



```python
def evaluate(node):
    if isinstance(node, Num):
        return node.value
    left = evaluate(node.left)
    right = evaluate(node.right)
    match node.op:
        case "+": result = left + right
        case "*": result = left * right
        case "^": result = left * right
     return result
```



```python
def evaluate(node):
    if isinstance(node, Num):
        return node.value
    left = evaluate(node.left)
    right = evaluate(node.right)
    match node.op:
        case "+": result = left + right
        case "*": result = left * right
        case "^": result = left * right
     return result
```

Streams — evaluate(node)

node.op	left	right	result
^	4	2	8
*	3	8	24
+	2	24	26

Variables — evaluate(node) · invocation 1 of 3

node	BinOp
left	4
right	2
result	8

Data-Oriented Debugging Across Place and Time

Questions?

Jeffrey Tao

https://speculative.tech

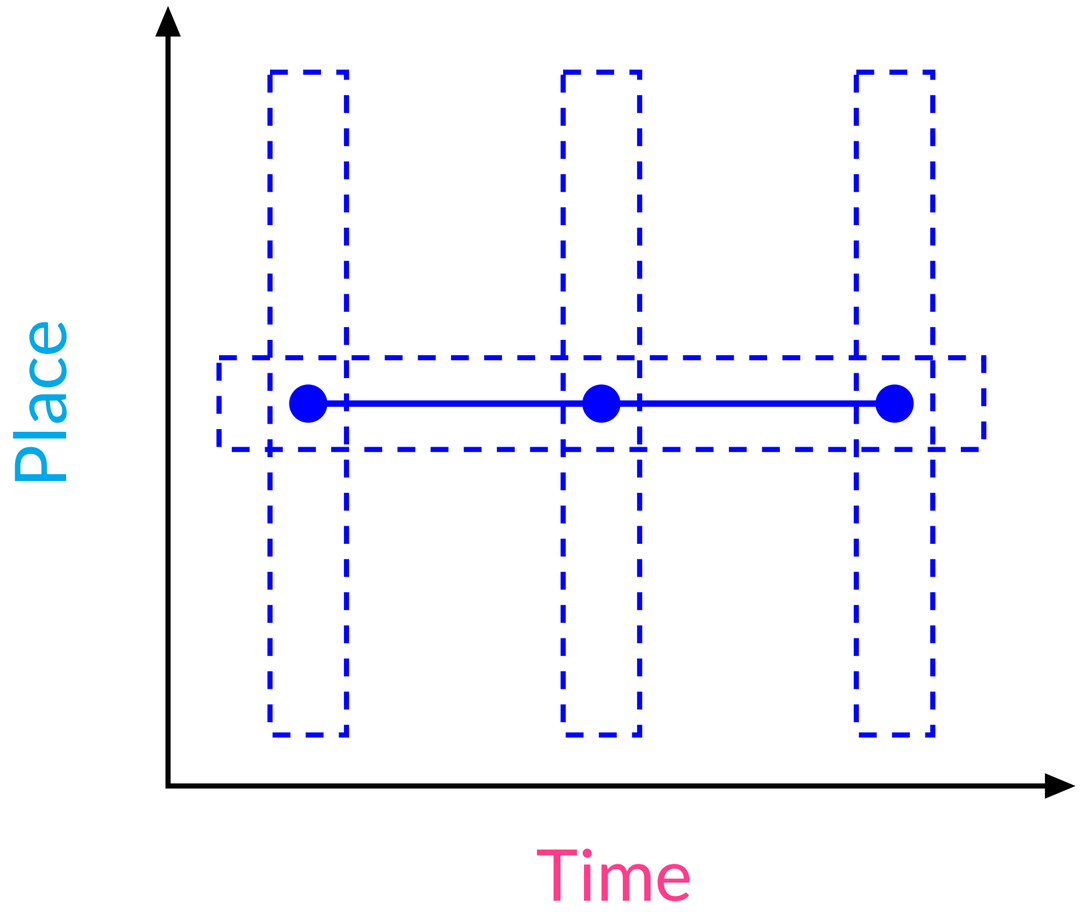
Appendix

🚨 Break glass in case of AI question

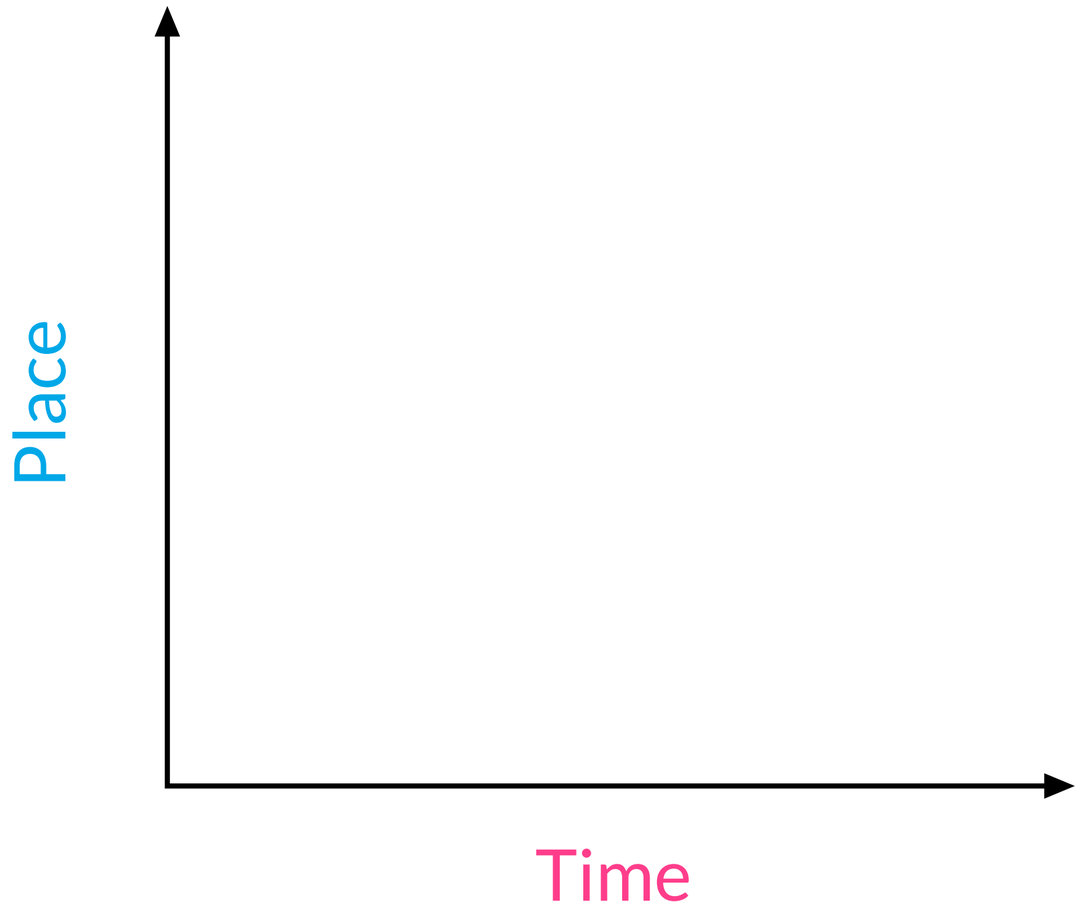
Hasn't AI made debugging interfaces obsolete?

- Data-Oriented Debugging concerns *responses to data needs*
- AI coding agents do better with objective feedback
- Debugging is still an interesting setting for observing human comprehension of complex systems

autopsy

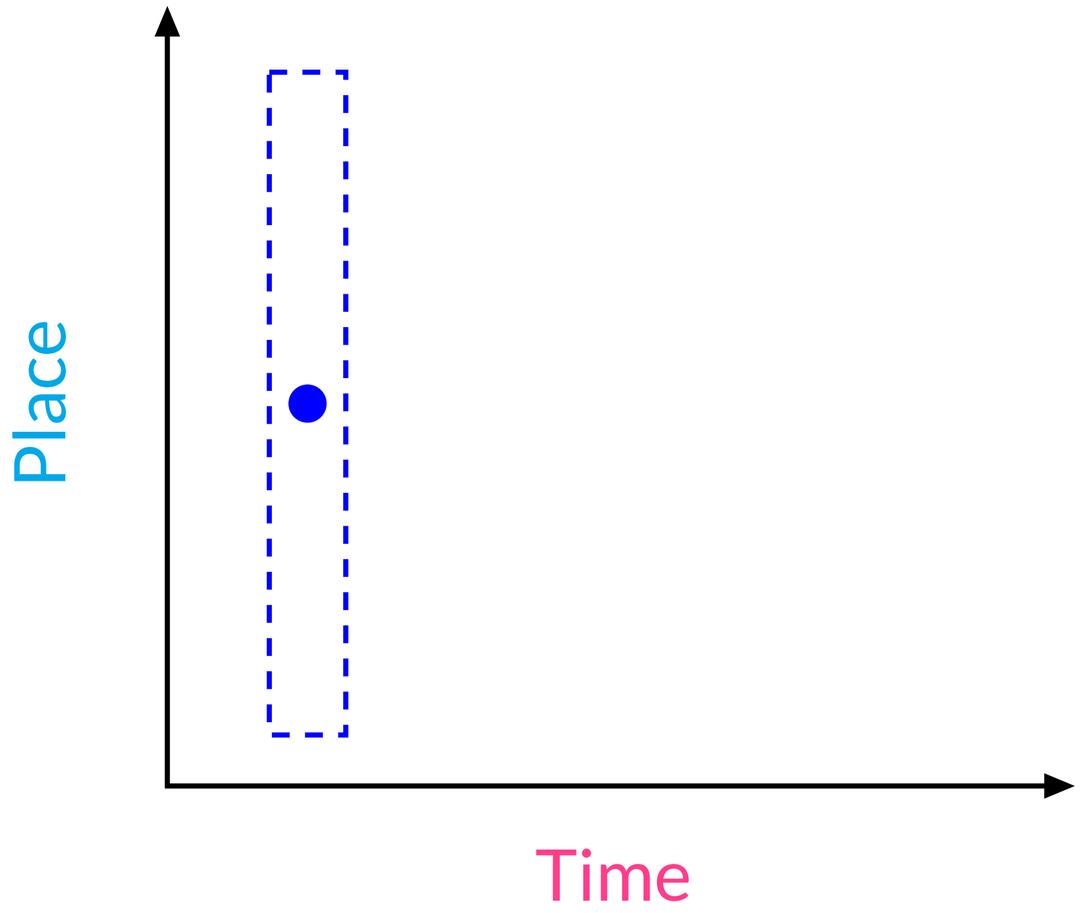


autopsy



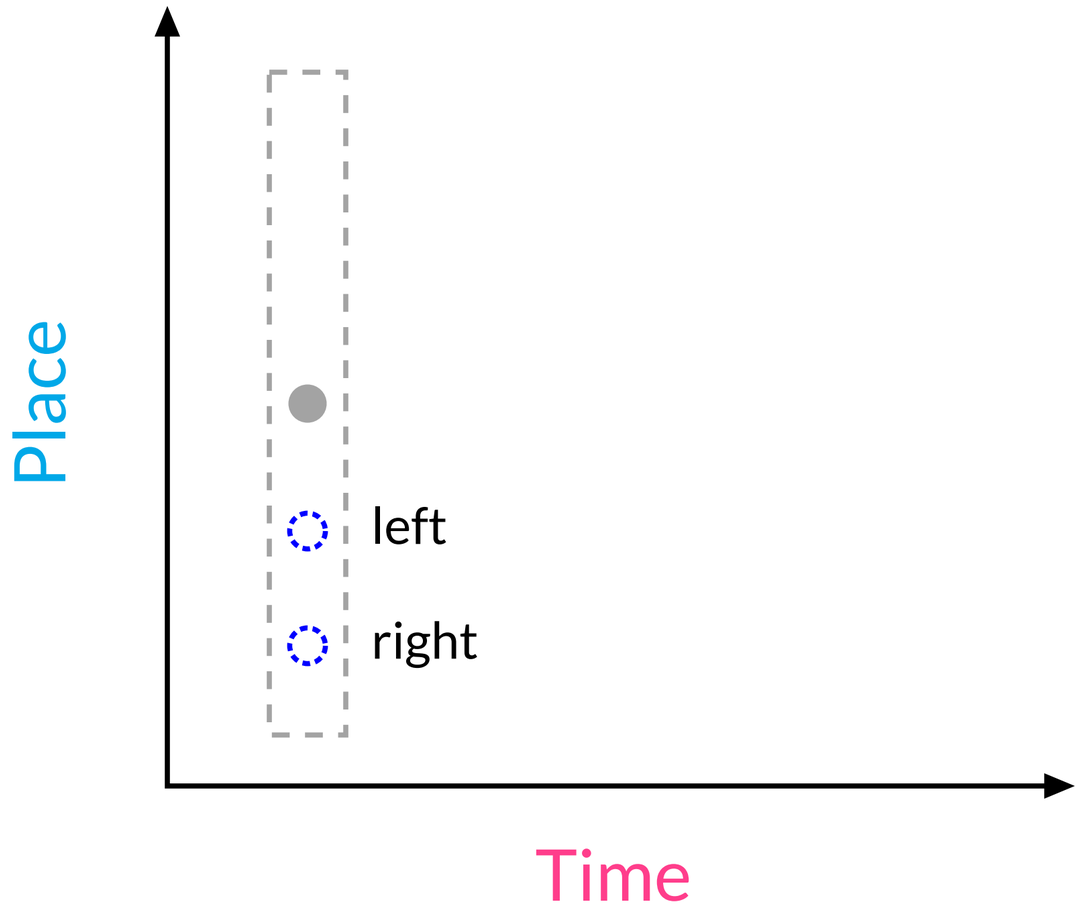

```
1 def evaluate(node):
2     if isinstance(node, Num):
3         return node.value
4     left = evaluate(node.left)
5     right = evaluate(node.right)
6     match node.op:
7         case "+": result = left + right
8         case "*": result = left * right
9         case "^": result = left * right
10     log(node.op)
11     return result
```

autopsy



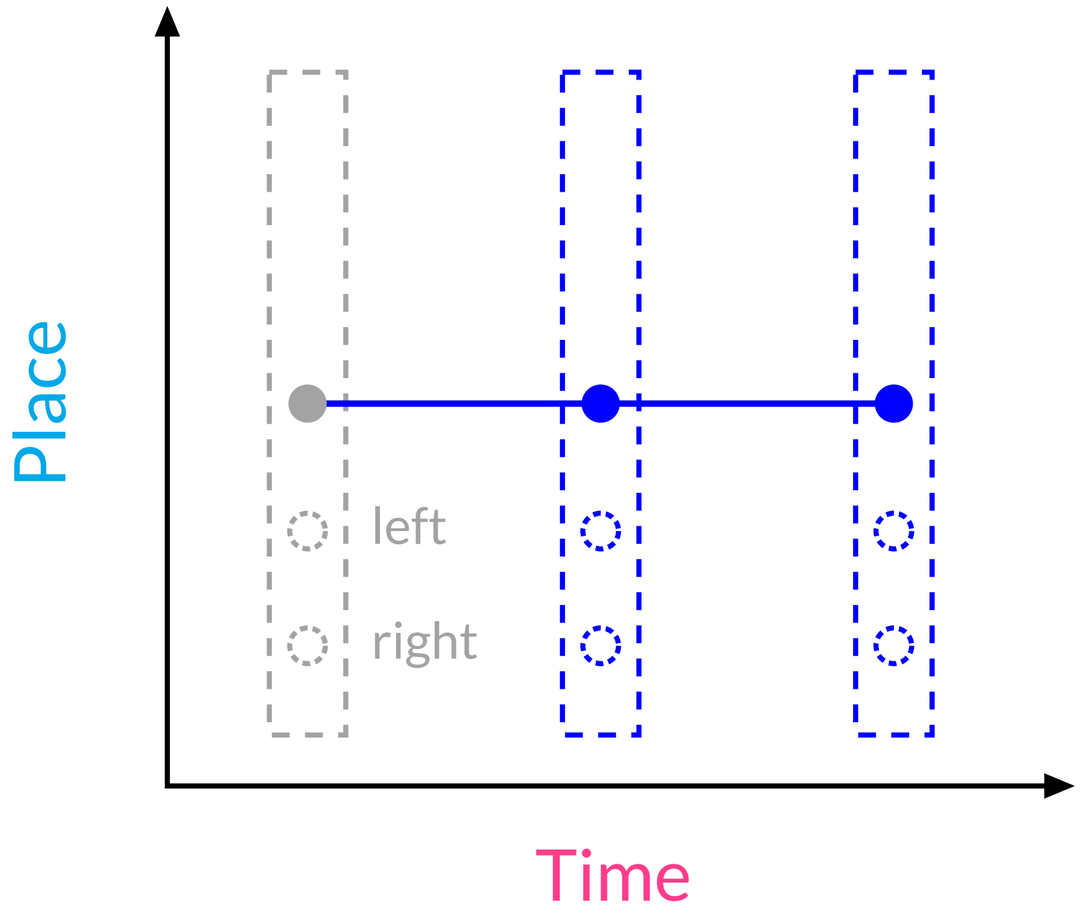

```
1 def evaluate(node):
2     if isinstance(node, Num):
3         return node.value
4     left = evaluate(node.left)
5     right = evaluate(node.right)
6     match node.op:
7         case "+": result = left + right
8         case "*": result = left * right
9         case "^": result = left * right
10     log(node.op)
11     return result
```

autopsy



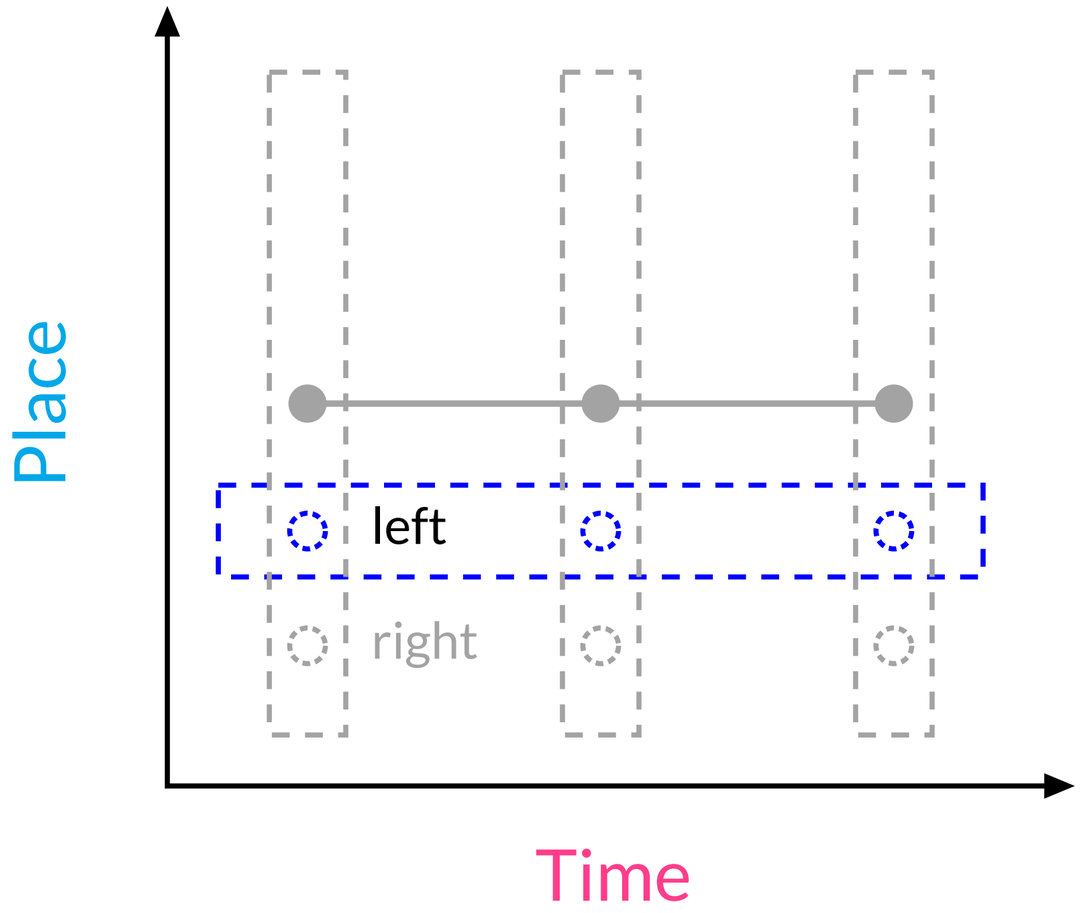

```
1 def evaluate(node):
2     if isinstance(node, Num):
3         return node.value
4     left = evaluate(node.left)
5     right = evaluate(node.right)
6     match node.op:
7         case "+": result = left + right
8         case "*": result = left * right
9         case "^": result = left * right
10     log(node.op)
11     return result
```

autopsy



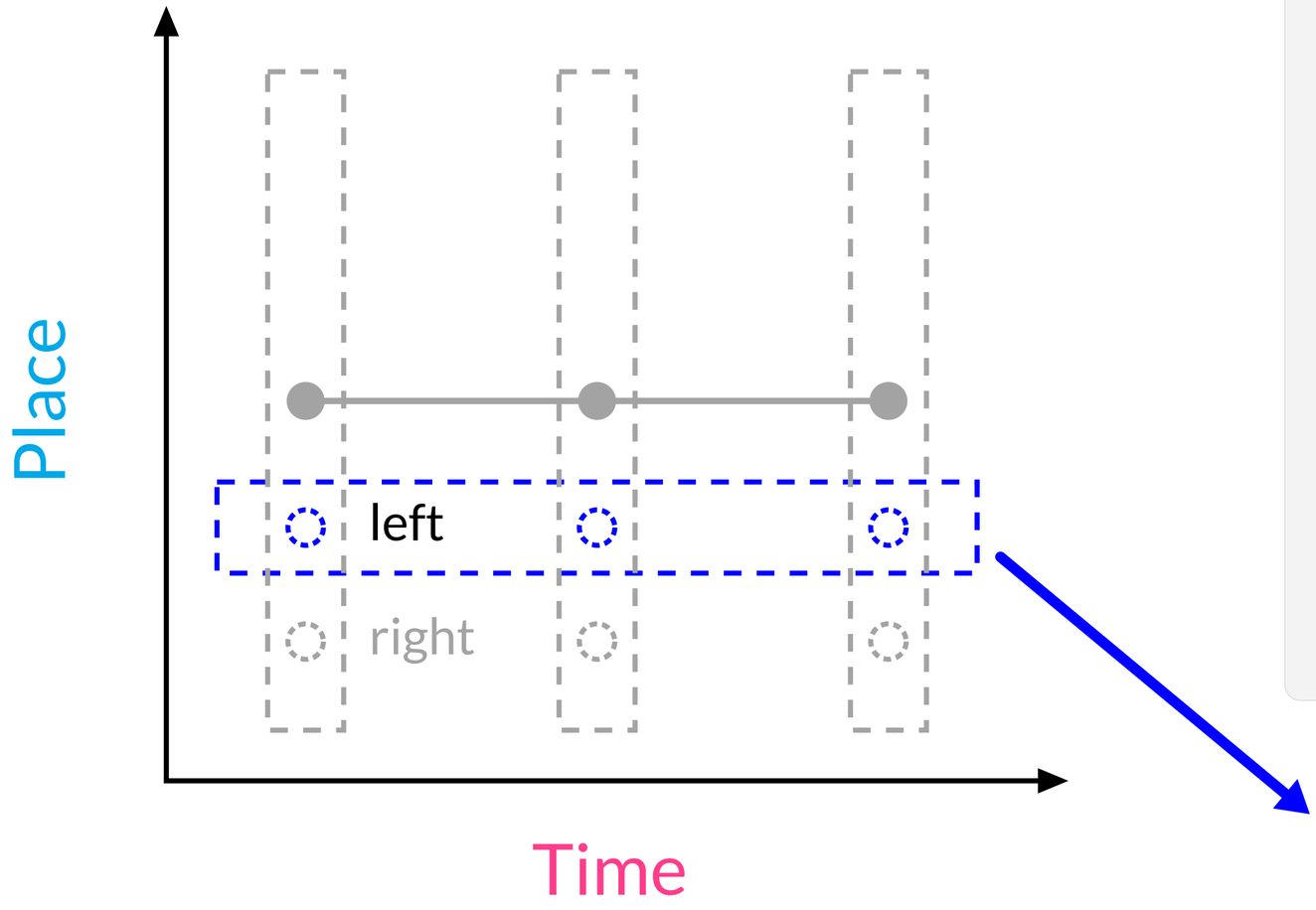

```
1 def evaluate(node):
2     if isinstance(node, Num):
3         return node.value
4     left = evaluate(node.left)
5     right = evaluate(node.right)
6     match node.op:
7         case "+": result = left + right
8         case "*": result = left * right
9         case "^": result = left * right
10     log(node.op)
11     return result
```

autopsy




```
1 def evaluate(node):
2     if isinstance(node, Num):
3         return node.value
4     left = evaluate(node.left)
5     right = evaluate(node.right)
6     match node.op:
7         case "+": result = left + right
8         case "*": result = left * right
9         case "^": result = left * right
10     log(node.op)
11     return result
```

autopsy




```
1 def evaluate(node):
2     if isinstance(node, Num):
3         return node.value
4     left = evaluate(node.left)
5     right = evaluate(node.right)
6     match node.op:
7         case "+": result = left + right
8         case "*": result = left * right
9         case "^": result = left * right
10     log(node.op)
11     return result
```

min, avg, max, ...